• PROJECT SENTINEL / 3D HOLOGRAPHIC GLOBE HUD

JARVIS EPISODE 30 GUIDE

by Pulse of AI

▲ CRITICAL WARNING: MANDATORY PRE-REQUISITE

You must watch all the previous videos (Parts 1 to 29) and extract the baseline code and environment files directly from their pinned comments. If you do not configure the previous dependencies, JARVIS might not work!

01. COMPLETE SERIES ROADMAP (WATCH IN CHRONOLOGICAL ORDER)

MODULE	ENGINEERING CAPABILITY ADDED	DIRECT SOURCE
PART 1	Core Brain Setup & Instant Live API Integration	■ Watch Ep 1
PART 2	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 2
PART 3	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 3
PART 4	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 4
PART 5	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 5
PART 6	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 6
PART 7	Autonomous Desktop Tool Calling Active	■ Watch Ep 7
PART 8	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 8
PART 9	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 9
PART 10	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 10
PART 11	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 11
PART 12	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 12
PART 13	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 13
PART 14	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 14
PART 15	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 15
PART 16	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 16
PART 17	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 17
PART 18	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 18
PART 19	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 19

PART 20	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 20
PART 21	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 21
PART 22	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 22
PART 23	Autonomous Serious Mode Pipeline	■ Watch Ep 23
PART 24	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 24
PART 25	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 25
PART 26	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 26
PART 27	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 27
PART 28	Autonomous System Expansion & Pipeline Upgrades	■ Watch Ep 28
PART 29	Project Doppelgänger (Ghost Operator)	■ Watch Ep 29
PART 30	**Project Sentinel (3D Holographic Globe HUD)**	■ Watch Ep 30

02. ENVIRONMENT ARCHITECTURE & SYSTEM CONFIG

[1] Terminal Dependencies:
pip install google-genai pyaudio geopy plotly PyQt6 PyQt6-WebEngine flask pywebview requests

[2] Secure Environment File (.env):
GOOGLE_API_KEY=your_gemini_api_key

[3] Episode 30 Core Innovation:
Integration of a CesiumJS web engine powered by a Flask backend and pywebview to render a borderless, interactive 3D holographic Earth widget with real-time geodesic flight paths.

03. PRODUCTION CODEBASE (DYNAMICALLY EXTRACTED)

The following source code has been extracted directly from your local project files.

// MODULE: globe_hud.py (PROJECT SENTINEL 3D GLOBE ENGINE (CESIUMJS/FLASK))

```
# -*- coding: utf-8 -*-
"""
globe_hud.py — JARVIS :: Project Sentinel (v2 — CesiumJS Edition)
Real-satellite-imagery 3D Earth with animated neon trajectory arcs.
Uses CesiumJS (NASA/SpaceX-grade globe engine) + ESRI World Imagery tiles
(no API key, Google Earth quality).

Architecture
■■■■■■■■■■■■■
 Python ■■[Flask:9191]■■■ HTML/CesiumJS in pywebview subprocess
 agent.py calls launch_globe_hud(origin, destination) as a tool.

Built for Swayam Dhawale — Pulse of AI.
"""

import os
import sys
import math
import json
import time
import threading
import webbrowser
import subprocess
import logging
from typing import Optional

# ■■ Optional imports ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
try:
    import requests as _requests
    _HAS_REQUESTS = True
except ImportError:
    _HAS_REQUESTS = False

try:
    import webview as _webview
    _HAS_WEBVIEW = True
except ImportError:
    _HAS_WEBVIEW = False

try:
    from flask import Flask, jsonify, render_template_string, request as _flask_req
    _HAS_FLASK = True
except ImportError:
    _HAS_FLASK = False

# ■■ Constants ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
FLASK_PORT = 9292 # Separate port from orbital_hud to avoid conflicts
_flask_thread: Optional[threading.Thread] = None
_server_started = False
_current_route: dict = {}

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# GEOCODING ENGINE
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

# Built-in city database — instant, no network for common cities
_CITIES = {
    "new york": (40.7128, -74.0060, "New York"),
    "new york city": (40.7128, -74.0060, "New York"),
    "nyc": (40.7128, -74.0060, "New York"),
    "london": (51.5074, -0.1278, "London"),
```

```
    "paris": (48.8566, 2.3522, "Paris"),
    "tokyo": (35.6762, 139.6503, "Tokyo"),
    "beijing": (39.9042, 116.4074, "Beijing"),
    "shanghai": (31.2304, 121.4737, "Shanghai"),
    "dubai": (25.2048, 55.2708, "Dubai"),
    "singapore": (1.3521, 103.8198, "Singapore"),
    "sydney": (-33.8688, 151.2093, "Sydney"),
    "los angeles": (34.0522, -118.2437, "Los Angeles"),
    "la": (34.0522, -118.2437, "Los Angeles"),
    "chicago": (41.8781, -87.6298, "Chicago"),
    "san francisco": (37.7749, -122.4194, "San Francisco"),
    "sf": (37.7749, -122.4194, "San Francisco"),
    "moscow": (55.7558, 37.6173, "Moscow"),
    "berlin": (52.5200, 13.4050, "Berlin"),
    "rome": (41.9028, 12.4964, "Rome"),
    "madrid": (40.4168, -3.7038, "Madrid"),
    "toronto": (43.6532, -79.3832, "Toronto"),
    "mumbai": (19.0760, 72.8777, "Mumbai"),
    "delhi": (28.7041, 77.1025, "Delhi"),
    "new delhi": (28.6139, 77.2090, "New Delhi"),
    "bangalore": (12.9716, 77.5946, "Bangalore"),
    "kolkata": (22.5726, 88.3639, "Kolkata"),
    "chennai": (13.0827, 80.2707, "Chennai"),
    "hyderabad": (17.3850, 78.4867, "Hyderabad"),
    "pune": (18.5204, 73.8567, "Pune"),
    "ahmedabad": (23.0225, 72.5714, "Ahmedabad"),
    "cairo": (30.0444, 31.2357, "Cairo"),
    "johannesburg": (-26.2041, 28.0473, "Johannesburg"),
    "nairobi": (-1.2921, 36.8219, "Nairobi"),
    "istanbul": (41.0082, 28.9784, "Istanbul"),
    "bangkok": (13.7563, 100.5018, "Bangkok"),
    "hong kong": (22.3193, 114.1694, "Hong Kong"),
    "seoul": (37.5665, 126.9780, "Seoul"),
    "amsterdam": (52.3676, 4.9041, "Amsterdam"),
    "zurich": (47.3769, 8.5417, "Zurich"),
    "vienna": (48.2082, 16.3738, "Vienna"),
    "stockholm": (59.3293, 18.0686, "Stockholm"),
    "oslo": (59.9139, 10.7522, "Oslo"),
    "buenos aires": (-34.6037, -58.3816, "Buenos Aires"),
    "sao paulo": (-23.5505, -46.6333, "Sao Paulo"),
    "mexico city": (19.4326, -99.1332, "Mexico City"),
    "seattle": (47.6062, -122.3321, "Seattle"),
    "boston": (42.3601, -71.0589, "Boston"),
    "miami": (25.7617, -80.1918, "Miami"),
    "atlanta": (33.7490, -84.3880, "Atlanta"),
    "dallas": (32.7767, -96.7970, "Dallas"),
    "houston": (29.7604, -95.3698, "Houston"),
    "phoenix": (33.4484, -112.0740, "Phoenix"),
    "denver": (39.7392, -104.9903, "Denver"),
    "washington": (38.9072, -77.0369, "Washington DC"),
    "washington dc": (38.9072, -77.0369, "Washington DC"),
    "taipei": (25.0330, 121.5654, "Taipei"),
    "jakarta": (-6.2088, 106.8456, "Jakarta"),
    "kuala lumpur": (3.1390, 101.6869, "Kuala Lumpur"),
    "kl": (3.1390, 101.6869, "Kuala Lumpur"),
    "manila": (14.5995, 120.9842, "Manila"),
    "karachi": (24.8607, 67.0011, "Karachi"),
    "lahore": (31.5204, 74.3587, "Lahore"),
    "dhaka": (23.8103, 90.4125, "Dhaka"),
    "tehran": (35.6892, 51.3890, "Tehran"),
    "baghdad": (33.3152, 44.3661, "Baghdad"),
    "riyadh": (24.7136, 46.6753, "Riyadh"),
    "abu dhabi": (24.4539, 54.3773, "Abu Dhabi"),
    "doha": (25.2854, 51.5310, "Doha"),
    "cape town": (-33.9249, 18.4241, "Cape Town"),
    "lagos": (6.5244, 3.3792, "Lagos"),
    "accra": (5.6037, -0.1870, "Accra"),
    "casablanca": (33.5731, -7.5898, "Casablanca"),
```

```
    "addis ababa": (9.0320, 38.7469, "Addis Ababa"),
    "dar es salaam": (-6.7924, 39.2083, "Dar es Salaam"),
    "lima": (-12.0464, -77.0428, "Lima"),
    "bogota": (4.7110, -74.0721, "Bogota"),
    "santiago": (-33.4489, -70.6693, "Santiago"),
    "caracas": (10.4806, -66.9036, "Caracas"),
    "montreal": (45.5017, -73.5673, "Montreal"),
    "vancouver": (49.2827, -123.1207, "Vancouver"),
    "calgary": (51.0447, -114.0719, "Calgary"),
    "melbourne": (-37.8136, 144.9631, "Melbourne"),
    "brisbane": (-27.4698, 153.0251, "Brisbane"),
    "perth": (-31.9505, 115.8605, "Perth"),
    "auckland": (-36.8485, 174.7633, "Auckland"),
    "munich": (48.1351, 11.5820, "Munich"),
    "frankfurt": (50.1109, 8.6821, "Frankfurt"),
    "hamburg": (53.5753, 10.0153, "Hamburg"),
    "brussels": (50.8503, 4.3517, "Brussels"),
    "lisbon": (38.7223, -9.1393, "Lisbon"),
    "warsaw": (52.2297, 21.0122, "Warsaw"),
    "prague": (50.0755, 14.4378, "Prague"),
    "budapest": (47.4979, 19.0402, "Budapest"),
    "bucharest": (44.4268, 26.1025, "Bucharest"),
    "kyiv": (50.4501, 30.5234, "Kyiv"),
    "kiev": (50.4501, 30.5234, "Kyiv"),
    "athens": (37.9838, 23.7275, "Athens"),
    "tel aviv": (32.0853, 34.7818, "Tel Aviv"),
    "beirut": (33.8938, 35.5018, "Beirut"),
    "amman": (31.9454, 35.9284, "Amman"),
    "islamabad": (33.6844, 73.0479, "Islamabad"),
    "kathmandu": (27.7172, 85.3240, "Kathmandu"),
    "colombo": (6.9271, 79.8612, "Colombo"),
    "yangon": (16.8661, 96.1951, "Yangon"),
    "hanoi": (21.0278, 105.8342, "Hanoi"),
    "ho chi minh": (10.8231, 106.6297, "Ho Chi Minh City"),
    "ho chi minh city": (10.8231, 106.6297, "Ho Chi Minh City"),
    "phnom penh": (11.5564, 104.9282, "Phnom Penh"),
    "dhaka": (23.8103, 90.4125, "Dhaka"),
    "ulaanbaatar": (47.8864, 106.9057, "Ulaanbaatar"),
    "tashkent": (41.2995, 69.2401, "Tashkent"),
    "baku": (40.4093, 49.8671, "Baku"),
    "tbilisi": (41.7151, 44.8271, "Tbilisi"),
    "yerevan": (40.1792, 44.4991, "Yerevan"),
    "almaty": (43.2220, 76.8512, "Almaty"),
}


def geocode_location(name: str) -> tuple:
    """Returns (lat, lon, display_name). Fast built-in DB + Nominatim fallback."""
    key = name.strip().lower()
    if key in _CITIES:
        return _CITIES[key]

    if _HAS_REQUESTS:
        try:
            r = _requests.get(
                "https://nominatim.openstreetmap.org/search",
                params={"q": name, "format": "json", "limit": 1},
                headers={"User-Agent": "JARVIS-Sentinel/2.0"},
                timeout=7,
            )
            r.raise_for_status()
            data = r.json()
            if data:
                lat = float(data[0]["lat"])
                lon = float(data[0]["lon"])
                disp = data[0].get("display_name", name).split(",")[0].strip()
                return lat, lon, disp
        except Exception:
```

```
            pass

    raise ValueError(f"Cannot resolve location: '{name}'")


def haversine_km(lat1, lon1, lat2, lon2) -> float:
    """Great-circle distance in kilometres."""
    R = 6371.0
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp = math.radians(lat2 - lat1)
    dl = math.radians(lon2 - lon1)
    a = math.sin(dp/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(dl/2)**2
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))


def flight_time_str(km: float) -> str:
    """Rough commercial flight time at 900 km/h."""
    h = int(km / 900)
    m = int((km / 900 - h) * 60)
    return f"{h}h {m:02d}m" if h else f"{m}m"


# ═══════════════════════════════════════════════════════════════════════════
# FLASK SERVER
# ═══════════════════════════════════════════════════════════════════════════

def _make_app():
    app = Flask(__name__)

    @app.route("/")
    def index():
        return render_template_string(_HTML)

    @app.route("/api/route")
    def get_route():
        return jsonify(_current_route)

    @app.route("/api/set_route", methods=["POST"])
    def set_route():
        global _current_route
        _current_route = _flask_req.get_json(force=True) or {}
        return jsonify({"ok": True})

    return app


def _run_server():
    global _server_started
    if not _HAS_FLASK:
        return
    logging.getLogger("werkzeug").setLevel(logging.ERROR)
    _server_started = True
    _make_app().run(host="127.0.0.1", port=FLASK_PORT,
                    debug=False, use_reloader=False, threaded=True)


def _ensure_server():
    global _flask_thread, _server_started
    if _server_started:
        return
    if not _HAS_FLASK:
        return
    _flask_thread = threading.Thread(target=_run_server, daemon=True)
    _flask_thread.start()
    deadline = time.time() + 5.0
    while time.time() < deadline:
        try:
            _requests.get(f"http://127.0.0.1:{FLASK_PORT}/api/route", timeout=0.4)
```

```
            break
        except Exception:
            time.sleep(0.1)


# ═══════════════════════════════════════════════════════════════════════════
# PUBLIC API ← called by agent.py
# ═══════════════════════════════════════════════════════════════════════════

def launch_globe_hud(origin: str = "", destination: str = "", auto_open: bool = True) -> str:
    """
    JARVIS TOOL — Project Sentinel.
    Deploys a CesiumJS Google-Earth-quality 3D globe with a glowing
    trajectory arc between two global locations.

    Args:
        origin: e.g. 'Mumbai'
        destination: e.g. 'New York'
        auto_open: open the window immediately

    Returns:
        Status string for JARVIS to speak.
    """
    global _current_route
    payload: dict = {}

    if origin and destination:
        try:
            lat1, lon1, n1 = geocode_location(origin)
            lat2, lon2, n2 = geocode_location(destination)
            km = haversine_km(lat1, lon1, lat2, lon2)
            payload = {
                "origin": {"lat": lat1, "lon": lon1, "name": n1},
                "destination": {"lat": lat2, "lon": lon2, "name": n2},
                "distance_km": round(km, 1),
                "distance_mi": round(km * 0.621371, 1),
                "flight_time": flight_time_str(km),
            }
            _current_route = payload
        except ValueError as e:
            return f"GEOCODING ERROR: {e}"

    if auto_open:
        _open_window(payload)

    if payload:
        n1 = payload["origin"]["name"]
        n2 = payload["destination"]["name"]
        km = payload["distance_km"]
        ft = payload["flight_time"]
        return (
            f"SUCCESS: Sentinel Globe deployed. "
            f"Trajectory arc: {n1} to {n2}. "
            f"Distance: {km:,} km. Flight time: {ft}."
        )
    return "SUCCESS: JARVIS Sentinel Globe deployed."


def _open_window(payload: dict):
    _ensure_server()

    if payload and _HAS_REQUESTS and _server_started:
        try:
            _requests.post(
                f"http://127.0.0.1:{FLASK_PORT}/api/set_route",
                json=payload, timeout=2,
            )
        except Exception:
```

```
            pass

    viewer_py = os.path.join(os.path.dirname(os.path.abspath(__file__)), "globe_viewer.py")
    if _HAS_WEBVIEW and os.path.exists(viewer_py):
        try:
            flags = subprocess.CREATE_NO_WINDOW if sys.platform == "win32" else 0
            subprocess.Popen([sys.executable, viewer_py, str(FLASK_PORT)],
                             creationflags=flags)
            return
        except Exception as e:
            print(f"[Sentinel] viewer subprocess failed: {e}")

    webbrowser.open(f"http://127.0.0.1:{FLASK_PORT}/")


# ═══════════════════════════════════════════════════════════════════════
# HTML / CESIUMJS TEMPLATE
# ═══════════════════════════════════════════════════════════════════════

_HTML = r"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width,initial-scale=1.0"/>
<title>JARVIS :: SENTINEL</title>

<!-- CesiumJS 1.104 — battle-tested, stable, full feature set -->
<script src="https://cesium.com/downloads/cesiumjs/releases/1.104/Build/Cesium/Cesium.js"></script>
<link href="https://cesium.com/downloads/cesiumjs/releases/1.104/Build/Cesium/Widgets/widgets.css"
rel="stylesheet"/>

<style>
/* ═══════════════════════════════════════════
   JARVIS SENTINEL — Master Stylesheet
   ═══════════════════════════════════════════ */
:root {
  --c: #00e5ff; /* primary cyan */
  --c2: #0af; /* mid cyan */
  --b: #0066ff; /* blue */
  --g: #00ff88; /* green */
  --r: #ff3860; /* red */
  --bg: #00060d; /* near-black bg */
  --p: rgba(0,14,24,0.82); /* panel bg */
  --br: rgba(0,229,255,0.22); /* border */
  --tx: #8dd8ef; /* body text */
  --wh: #dff5ff; /* near-white */
}

* { margin:0; padding:0; box-sizing:border-box; }

html, body {
  width:100%; height:100%;
  background: var(--bg);
  font-family: 'Courier New', monospace;
  color: var(--tx);
  overflow: hidden;
  user-select: none;
}

/* ═══ Cesium fills entire viewport ═══ */
#cesiumContainer {
  position: fixed;
  inset: 36px 0 32px 0;
  background: #000;
}

/* Strip ALL Cesium built-in chrome */
.cesium-widget-credits,
```

```
.cesium-credit-lightbox-overlay,
.cesium-credit-textContainer { display:none !important; }
.cesium-viewer-toolbar { display:none !important; }
.cesium-viewer-animationContainer { display:none !important; }
.cesium-viewer-timelineContainer { display:none !important; }
.cesium-viewer-bottom { display:none !important; }

/* ■■ DRAG / TITLE BAR ■■ */
#titlebar {
  position:fixed; top:0; left:0; right:0; height:36px; z-index:1000;
  background: linear-gradient(90deg,
    #000a12 0%, rgba(0,229,255,0.07) 45%, rgba(0,102,255,0.07) 55%, #000a12 100%);
  border-bottom: 1px solid var(--br);
  display:flex; align-items:center; justify-content:space-between;
  padding: 0 14px;
  -webkit-app-region: drag; cursor:move;
}
.tb-logo {
  display:flex; align-items:center; gap:10px;
}
.tb-logo .hex {
  width:20px; height:20px;
  background: var(--c);
  clip-path: polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0% 25%);
  animation: hex-pulse 2s ease-in-out infinite;
}
@keyframes hex-pulse {
  0%,100%{ box-shadow:0 0 0 0 rgba(0,229,255,0.6); }
  50% { box-shadow:0 0 0 6px rgba(0,229,255,0); }
}
.tb-title {
  font-size:11px; letter-spacing:5px;
  color:var(--c); text-shadow:0 0 10px var(--c);
  font-weight:bold; text-transform:uppercase;
}
.tb-sub {
  font-size:9px; letter-spacing:3px; color:rgba(0,229,255,0.4);
  margin-top:1px;
}
.win-btns {
  display:flex; gap:8px; -webkit-app-region:no-drag;
}
.wb {
  width:13px; height:13px; border-radius:50%; border:none;
  cursor:pointer; transition:filter 0.2s, transform 0.15s;
}
.wb:hover { filter:brightness(1.5); transform:scale(1.2); }
.wb.close { background:#ff5f57; }
.wb.min { background:#febc2e; }

/* ■■ SCANLINES ■■ */
#scanlines {
  position:fixed; inset:36px 0 32px 0;
  pointer-events:none; z-index:50;
  background: repeating-linear-gradient(
    0deg, transparent, transparent 3px,
    rgba(0,229,255,0.009) 3px, rgba(0,229,255,0.009) 4px);
}

/* ■■ CORNER BRACKETS ■■ */
.corner {
  position:fixed; width:36px; height:36px;
  pointer-events:none; z-index:300;
}
.corner.tl { top:44px; left:8px; border-top:2px solid var(--c); border-left:2px solid var(--c); }
.corner.tr { top:44px; right:8px; border-top:2px solid var(--c); border-right:2px solid var(--c); }
.corner.bl { bottom:40px; left:8px; border-bottom:2px solid var(--c); border-left:2px solid var(--c); }
.corner.br { bottom:40px; right:8px; border-bottom:2px solid var(--c); border-right:2px solid var(--c); }
```

```
/* ■■ HUD PANELS ■■ */
#panel-l {
  position:fixed; left:14px; top:54px; bottom:44px;
  width:210px; z-index:200;
  display:flex; flex-direction:column; gap:8px;
  pointer-events:none;
}
#panel-r {
  position:fixed; right:14px; top:54px; bottom:44px;
  width:210px; z-index:200;
  display:flex; flex-direction:column; gap:8px;
  pointer-events:none;
}

/* ■■ CARD ■■ */
.card {
  background:var(--p);
  border:1px solid var(--br);
  border-radius:3px;
  padding:11px 13px;
  backdrop-filter: blur(16px);
  position:relative; overflow:hidden;
}
.card::after {
  content:'';
  position:absolute; inset:0;
  background: linear-gradient(135deg,
    rgba(0,229,255,0.025) 0%, transparent 60%);
  pointer-events:none;
}
.card::before {
  content:'';
  position:absolute; top:0; left:0; right:0; height:1px;
  background: linear-gradient(90deg, transparent 0%, var(--c) 50%, transparent 100%);
}
.c-title {
  font-size:8px; letter-spacing:3px; color:var(--c);
  text-transform:uppercase; opacity:0.7;
  margin-bottom:8px;
}
.c-val {
  font-size:21px; font-weight:bold; color:var(--wh);
  text-shadow:0 0 14px rgba(0,229,255,0.6);
  line-height:1.1;
}
.c-val .unit {
  font-size:11px; color:var(--c2); margin-left:3px; font-weight:normal;
}
.c-sub {
  font-size:9px; color:var(--tx); margin-top:4px;
  letter-spacing:1px; opacity:0.8;
}

/* ■■ ROUTE CARD ■■ */
.route-card {
  background:var(--p);
  border:1px solid rgba(0,229,255,0.35);
  border-radius:3px; padding:12px;
  backdrop-filter:blur(16px);
  position:relative; overflow:hidden;
}
.route-card::before {
  content:'';
  position:absolute; top:0; left:0; right:0; height:2px;
  background: linear-gradient(90deg, var(--b), var(--c), var(--b));
  background-size:200% 100%;
  animation:slide 3s linear infinite;
```

```
}
@keyframes slide {
  0% { background-position:200% 0; }
  100%{ background-position:-200% 0; }
}
.rl { font-size:8px; letter-spacing:2px; color:var(--c); opacity:0.65; text-transform:uppercase; }
.rc { font-size:14px; font-weight:bold; color:var(--wh);
      text-shadow:0 0 10px rgba(0,229,255,0.5);
      margin:2px 0 0 0; white-space:nowrap; overflow:hidden; text-overflow:ellipsis; }
.rarr {
  color:var(--c); font-size:16px; text-align:center;
  margin:5px 0;
  animation:parr 1.8s ease-in-out infinite;
}
@keyframes parr {
  0%,100%{ opacity:0.3; text-shadow:none; }
  50%{ opacity:1; text-shadow:0 0 12px var(--c); }
}

/* ■■ METRIC ROW ■■ */
.mrow {
  display:flex; justify-content:space-between; align-items:baseline;
  padding:4px 0;
  border-bottom:1px solid rgba(0,229,255,0.08);
}
.mrow:last-child { border:none; }
.ml { font-size:8px; letter-spacing:2px; color:var(--tx); opacity:0.6; }
.mv { font-size:12px; color:var(--wh); font-weight:bold; }
.mv.ok { color:var(--g); }
.mv.hot { color:var(--c); }

/* ■■ PROGRESS BAR ■■ */
.bar-wrap { margin-top:8px; }
.bar-lbl { font-size:8px; letter-spacing:2px; color:var(--tx); margin-bottom:3px; }
.bar-trk { height:2px; background:rgba(0,229,255,0.08); border-radius:1px; }
.bar-fill {
  height:100%;
  background:linear-gradient(90deg, var(--b), var(--c));
  border-radius:1px;
  box-shadow:0 0 6px var(--c);
  transition: width 1.5s ease;
}

/* ■■ STATUS BAR ■■ */
#statusbar {
  position:fixed; bottom:0; left:0; right:0; height:32px; z-index:1000;
  background: linear-gradient(90deg,
    #000a12 0%, rgba(0,229,255,0.05) 50%, #000a12 100%);
  border-top:1px solid var(--br);
  display:flex; align-items:center; justify-content:space-between;
  padding:0 16px;
  font-size:9px; letter-spacing:2px; color:rgba(0,229,255,0.45);
}
#clock-big {
  font-size:13px; color:var(--c); letter-spacing:2px;
  text-shadow:0 0 8px var(--c);
}

/* ■■ TRAJECTORY BADGE ■■ */
#traj-badge {
  position:fixed; bottom:44px; left:50%; transform:translateX(-50%);
  z-index:400; pointer-events:none;
  opacity:0; transition:opacity 0.7s;
}
#traj-badge.show { opacity:1; }
.badge-inner {
  background:rgba(0,229,255,0.07);
  border:1px solid var(--c);
```

```
  padding:7px 22px; border-radius:2px;
  font-size:10px; letter-spacing:3px; color:var(--c);
  text-shadow:0 0 12px var(--c);
  text-transform:uppercase; white-space:nowrap;
}

/* ■■ BOOT LOADER ■■ */
#loader {
  position:fixed; inset:0; background:var(--bg);
  display:flex; flex-direction:column;
  align-items:center; justify-content:center;
  z-index:9999; transition:opacity 0.9s;
}
#loader.gone { opacity:0; pointer-events:none; }
.ld-ring {
  width:72px; height:72px; border-radius:50%;
  border:2px solid transparent;
  border-top-color:var(--c); border-right-color:var(--b);
  animation:spin 0.9s linear infinite; margin-bottom:20px;
}
@keyframes spin{ to{ transform:rotate(360deg); } }
.ld-txt {
  font-size:9px; letter-spacing:5px; color:var(--c);
  animation:blink 1.1s ease-in-out infinite;
}
@keyframes blink { 0%,100%{opacity:0.2;} 50%{opacity:1;} }
.ld-progress {
  margin-top:16px; width:200px; height:1px;
  background:rgba(0,229,255,0.1); border-radius:1px; overflow:hidden;
}
.ld-progress-fill {
  height:100%;
  background:linear-gradient(90deg,var(--b),var(--c));
  width:0%; animation:prog 2.5s ease forwards;
}
@keyframes prog { to{width:100%;} }

/* ■■ RESPONSIVE ■■ */
@media(max-width:900px){ #panel-l,#panel-r{ display:none; } }
</style>
</head>

<body>

<!-- BOOT LOADER -->
<div id="loader">
  <div class="ld-ring"></div>
  <div class="ld-txt">INITIALIZING SENTINEL GLOBE</div>
  <div class="ld-progress"><div class="ld-progress-fill"></div></div>
</div>

<!-- TITLE BAR -->
<div id="titlebar">
  <div class="tb-logo">
    <div class="hex"></div>
    <div>
      <div class="tb-title">JARVIS :: SENTINEL GLOBE</div>
      <div class="tb-sub">PROJECT SENTINEL — GLOBAL TRAJECTORY SYSTEM</div>
    </div>
  </div>
  <div class="win-btns">
    <button class="wb min" onclick="minimizeWindow()"></button>
    <button class="wb close" onclick="closeWindow()"></button>
  </div>
</div>

<!-- CORNER BRACKETS -->
<div class="corner tl"></div>
```

```
<div class="corner tr"></div>
<div class="corner bl"></div>
<div class="corner br"></div>

<!-- SCANLINES -->
<div id="scanlines"></div>

<!-- CESIUM GLOBE -->
<div id="cesiumContainer"></div>

<!-- LEFT PANEL -->
<div id="panel-l">

  <div class="route-card" id="route-card" style="display:none">
    <div class="rl">TRAJECTORY ORIGIN</div>
    <div class="rc" id="p-origin">—</div>
    <div class="rarr">&#9660; &#9660; &#9660;</div>
    <div class="rl">DESTINATION</div>
    <div class="rc" id="p-dest">—</div>
  </div>

  <div class="card" id="dist-card" style="display:none">
    <div class="c-title">Great-Circle Distance</div>
    <div class="c-val"><span id="p-km">—</span><span class="unit">KM</span></div>
    <div class="c-sub" id="p-mi">—</div>
    <div class="bar-wrap">
      <div class="bar-lbl">RANGE COVERAGE</div>
      <div class="bar-trk"><div class="bar-fill" id="p-bar" style="width:0%"></div></div>
    </div>
  </div>

  <div class="card" id="ft-card" style="display:none">
    <div class="c-title">Estimated Flight Time</div>
    <div class="c-val" id="p-ft">—</div>
    <div class="c-sub">@ 900 KM/H CRUISE SPEED</div>
  </div>

</div>

<!-- RIGHT PANEL -->
<div id="panel-r">

  <div class="card">
    <div class="c-title">System Telemetry</div>
    <div class="mrow"><span class="ml">GLOBE ENGINE</span><span class="mv ok">CESIUM 1.104</span></div>
    <div class="mrow"><span class="ml">IMAGERY</span><span class="mv ok">ESRI WORLD SAT</span></div>
    <div class="mrow"><span class="ml">TRAJECTORY</span><span class="mv hot" id="arc-st">STANDBY</span></div>
    <div class="mrow"><span class="ml">PROJECTION</span><span class="mv">PERSPECTIVE</span></div>
    <div class="mrow"><span class="ml">TERRAIN</span><span class="mv ok">ACTIVE</span></div>
  </div>

  <div class="card">
    <div class="c-title">Navigation</div>
    <div class="c-sub" style="line-height:2; letter-spacing:1px; font-size:9px">
      LEFT-DRAG  → ORBIT<br>
      RIGHT-DRAG → PAN<br>
      SCROLL    → ZOOM<br>
      DOUBLE-CLICK → FLY TO<br>
      HOME     → RESET VIEW
    </div>
  </div>

  <div class="card">
    <div class="c-title">Local Time</div>
    <div id="clock-big">--:--:--</div>
    <div class="c-sub" id="p-date" style="margin-top:4px">—</div>
  </div>
```

```
</div>

<!-- TRAJECTORY BADGE -->
<div id="traj-badge">
  <div class="badge-inner" id="badge-txt">TRAJECTORY LOCK ACQUIRED</div>
</div>

<!-- STATUS BAR -->
<div id="statusbar">
  <span>SENTINEL GLOBE v2.0 :: CESIUM + ESRI WORLD IMAGERY</span>
  <span id="sb-fps">FPS: --</span>
  <span>BUILT BY SWAYAM DHAWALE :: PULSE OF AI</span>
</div>

<!-- ═══════════════════════════════════════════════════════════════
     CESIUM + JARVIS ENGINE
     ═══════════════════════════════════════════════════════════════ -->
<script>
'use strict';

// ── Window controls ──────────────────────────────────────────────
function closeWindow() { try{ pywebview.api.close(); }catch(e){ window.close(); } }
function minimizeWindow() { try{ pywebview.api.minimize(); }catch(e){} }

// ── Clock ────────────────────────────────────────────────────────
const DAYS = ['SUN','MON','TUE','WED','THU','FRI','SAT'];
const MONTHS = ['JAN','FEB','MAR','APR','MAY','JUN','JUL','AUG','SEP','OCT','NOV','DEC'];
function tick() {
  const n = new Date();
  const hh = String(n.getHours()).padStart(2,'0');
  const mm = String(n.getMinutes()).padStart(2,'0');
  const ss = String(n.getSeconds()).padStart(2,'0');
  document.getElementById('clock-big').textContent = `${hh}:${mm}:${ss}`;
  document.getElementById('p-date').textContent =
    `${DAYS[n.getDay()]} ${n.getDate()} ${MONTHS[n.getMonth()]} ${n.getFullYear()}`;
}
setInterval(tick, 1000); tick();

// ═══════════════════════════════════════════════════════════════
// CESIUM INITIALIZATION
// ═══════════════════════════════════════════════════════════════

// Suppress Ion token errors — we use ESRI, which is free and needs no token
Cesium.Ion.defaultAccessToken = '';

// ── Viewer setup ─────────────────────────────────────────────────
const viewer = new Cesium.Viewer('cesiumContainer', {

  // ESRI World Imagery — identical quality to Google Earth, 100% free
  imageryProvider: new Cesium.ArcGisMapServerImageryProvider({
    url: 'https://services.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer',
    enablePickFeatures: false,
  }),

  // Ellipsoid terrain (smooth, no token required)
  terrainProvider: new Cesium.EllipsoidTerrainProvider(),

  // Disable ALL default Cesium UI chrome
  baseLayerPicker: false,
  navigationHelpButton: false,
  animation: false,
  timeline: false,
  fullscreenButton: false,
  homeButton: false,
  geocoder: false,
  sceneModePicker: false,
  infoBox: false,
  selectionIndicator: false,
```

```
  creditContainer: document.createElement('div'), // discard credits UI

  // Performance
  requestRenderMode: false,
  maximumRenderTimeChange: Infinity,
});

const scene = viewer.scene;
const camera = viewer.camera;
const globe = scene.globe;

// ■■ Globe visual settings ■■■
```

```
 * @param {number} steps — number of arc segments
 * @param {number} maxAltKm — peak altitude in km (visible from orbit)
 * @returns {Cesium.Cartesian3[]}
 */
function buildArcPositions(lat1, lon1, lat2, lon2, steps=200, maxAltKm=900) {
  const start = new Cesium.Cartographic(
    Cesium.Math.toRadians(lon1), Cesium.Math.toRadians(lat1)
  );
  const end = new Cesium.Cartographic(
    Cesium.Math.toRadians(lon2), Cesium.Math.toRadians(lat2)
  );
  const geodesic = new Cesium.EllipsoidGeodesic(start, end);
  const positions = [];
  for (let i = 0; i <= steps; i++) {
    const t = i / steps;
    const mid = geodesic.interpolateUsingFraction(t);
    mid.height = Math.sin(t * Math.PI) * maxAltKm * 1000;
    positions.push(Cesium.Cartographic.toCartesian(mid));
  }
  return positions;
}

// ■■ Route state ■■■
```

```
// Build arc point array (200 steps, 900 km peak altitude)
_arcPositions = buildArcPositions(O.lat, O.lon, D.lat, D.lon, 200, 900);

// ■■ Origin marker ■■■
```

```
  // ■■ Travel dot ■■■
```

```
});

// ═══════════════════════════════════════════════════════════════════════════
// ROUTE DATA POLLING (from Flask backend)
// ═══════════════════════════════════════════════════════════════════════════
let _lastKey = '';

async function pollRoute() {
  try {
    const r = await fetch('/api/route');
    if (!r.ok) return;
    const data = await r.json();
    if (!data || !data.origin) return;
    const key = `${data.origin.name}|${data.destination.name}`;
    if (key !== _lastKey) {
      _lastKey = key;
      renderRoute(data);
    }
  } catch(e) {}
}

// Poll every 1.5 seconds for new routes pushed from Python
setInterval(pollRoute, 1500);

// ═══ Boot sequence ═══════════════════════════════════════════════════════
window.addEventListener('load', () => {
  // First poll immediately
  pollRoute();

  // Hide loader when Cesium globe is ready
  const hideLoader = () => {
    document.getElementById('loader').classList.add('gone');
  };
  // Try scene ready event first, fall back to timer
  if (scene.globe.tilesLoaded) {
    setTimeout(hideLoader, 1200);
  } else {
    const unsub = scene.globe.tileLoadProgressEvent.addEventListener(n => {
      if (n === 0) { unsub(); setTimeout(hideLoader, 600); }
    });
    setTimeout(hideLoader, 4000); // hard timeout
  }
});
</script>
</body>
</html>
"""

# ═══════════════════════════════════════════════════════════════════════════
# STANDALONE ENTRYPOINT
# ═══════════════════════════════════════════════════════════════════════════
if __name__ == "__main__":
    import sys as _sys
    _sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    origin_arg = _sys.argv[1] if len(_sys.argv) > 1 else "Mumbai"
    dest_arg = _sys.argv[2] if len(_sys.argv) > 2 else "New York"
    result = launch_globe_hud(origin_arg, dest_arg, auto_open=True)
    print(result)
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        pass
```

// MODULE: globe_viewer.py (PYWEBVIEW BORDERLESS WIDGET LAUNCHER)

```
"""
globe_viewer.py
Dedicated subprocess launcher so pywebview runs on its own main thread.
Larger window, more padding for Cesium globe.
Usage: python globe_viewer.py [port]
"""
import sys
import webview

def main():
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 9292
    url = f"http://127.0.0.1:{port}/"

    window = webview.create_window(
        title="JARVIS :: SENTINEL GLOBE",
        url=url,
        width=1280,
        height=860,
        resizable=True,
        frameless=True,
        transparent=False,
        on_top=True,
        background_color="#000608",
        text_select=False,
        min_size=(800, 600),
    )
    webview.start(debug=False, http_server=False, gui="edgechromium")

if __name__ == "__main__":
    main()
```

// MODULE: agent.py (CORE GEMINI LIVE ENGINE (UPDATED WITH GLOBE TOOL))

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD Console (Episode 23)
Autonomous App Orchestration, Multi-Tab Web Swarm, PPT/PDF Engine & Master Serious Mode Pipeline
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import time
import math
import struct
import threading
import subprocess
import collections
import queue
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
import requests
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console
import fui_bridge

load_dotenv()
if sys.platform == "win32":
    try:
```

```
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    except Exception:
        pass
console = Console()

# --- REMOTE KILL-SWITCH (Pulse of AI) ---
STATUS_URL = "https://gist.githubusercontent.com/PulseOfAIDev/a9951c6e858aacbabb604a0ed23f6ab7/raw/status.json"

def verify_app_access():
    """Checks remote status from GitHub Gist and displays cloud-configured message."""
    try:
        fresh_url = f"{STATUS_URL}?t={int(time.time())}"
        response = requests.get(fresh_url, headers={"Cache-Control": "no-cache"}, timeout=4).json()

        status = str(response.get("app_status", "free")).strip().lower()
        message = response.get(
            "message",
            "The free beta access period has concluded. To activate a subscription tier or upgrade your account,
visit: https://pulseofaidev.github.io"
        )

        if status != "free":
            console.print(f"\n[bold red]■ [ACCESS RESTRICTED][/bold red] {message}\n")
            sys.exit(0)
    except Exception:
        pass

# Run verification immediately on boot
verify_app_access()

# --- SECURE KEYRING INTEGRATION ---
from Setup import get_or_prompt_api_key

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)
```

```
# Self-Modifying Tri-Core Dynamic Skill Synthesizer
from skill_synthesizer import (
    evolve_skill,
    list_active_skills,
)

# Fast Cloud Reasoning Tool
from llm_router import (
    query_deepseek_reasoning,
)

# Episode 17: Autonomous App Composer & Typing Engine
from app_automator import (
    open_and_compose,
)

# Episode 18 & 19: Autonomous PowerPoint Generator & Presentation Engine
from ppt_engine import (
    create_and_launch_ppt,
)

# Episode 19 & 20: Autonomous Executive PDF Briefing Engine & Episode 23 Master Pipeline
from pdf_engine import (
    research_and_generate_pdf,
    execute_serious_mode,
)

# Episode 21: Autonomous Web Swarm & Browser Automation
from web_crawler import (
    run_autonomous_swarm,
)

# Episode 22: Local GPU Offline RAG Brain
from local_rag_engine import (
    process_dossier_offline,
)

# Episode 26: Autonomous Optical Physical Vision & Sentry Defense Core
from sentry_vision import (
    inspect_physical_world,
    activate_sentry_mode,
    deactivate_sentry_mode,
    get_sentry_security_report,
    capture_camera_snapshot,
    check_user_presence,
)

# Episode 27: Autonomous Screen Auto-Clicker & GUI Grounding Engine
from screen_clicker import (
    click_on_screen,
    double_click_on_screen,
    right_click_on_screen,
    move_cursor_to_element,
)

# Episode 28: Autonomous Desktop Pilot & Computer-Use Operator (Mark 85 Omni-Pilot)
from desktop_pilot import (
    execute_autonomous_desktop_pilot,
    abort_desktop_pilot,
)

# Episode 29: Autonomous Ambient Ghost Operator & Executive Shadow Radar (Project Doppelgänger)
from ghost_operator import (
    activate_ghost_mode,
    deactivate_ghost_mode,
    get_ghost_radar_status,
    prepare_meeting_briefing,
    apply_ghost_recommendation,
```

```
    analyze_clipboard_context,
    ghost_operator,
)

# Episode 30: 100% Local LLM & Autonomous Offline Sovereign Core (Project Sovereign)
from offline_engine import (
    offline_jarvis,
    activate_offline_mode,
    deactivate_offline_mode,
    get_offline_system_status,
    check_internet_connection,
)

# Project Orbital: 3D Holographic Earth HUD & Live Telemetry Grid
from orbital_hud import (
    launch_orbital_hud,
)

# Project Sentinel: Advanced 3D Globe HUD with Trajectory Arc Engine
from globe_hud import (
    launch_globe_hud,
)

# Dynamically initialize Gemini client using the secure keyring setup
client = genai.Client(api_key=get_or_prompt_api_key())

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
server_turn_active = False
is_user_speaking = False
discard_current_turn = False
acoustic_barge_in_enabled = True
media_is_playing_state = False
audio_player = None

# Dynamic Auto-Calibrated Audio Thresholds (tuned for natural voice sensitivity)
ambient_noise_p95 = 350.0
speech_start_threshold = 1100.0
speech_continue_threshold = 650.0
barge_in_threshold = 3800.0

class SmoothAudioPlayer:
    """Zero-latency, jitter-buffered background audio playback worker.
    Eliminates underflows, stuttering, and robotic audio lag on Windows.
    """
    def __init__(self, sample_rate=24000, buffer_frames=1024):
        self.sample_rate = sample_rate
        self.buffer_frames = buffer_frames
        self.audio_queue = queue.Queue()
        self.is_playing = False
        self.stop_flag = threading.Event()
        self.last_play_time = 0.0
        self.thread = threading.Thread(target=self._play_loop, daemon=True)
        self.thread.start()

    def _play_loop(self):
        try:
            stream = p.open(
                format=FORMAT,
                channels=CHANNELS,
                rate=self.sample_rate,
```

```
            output=True,
            frames_per_buffer=self.buffer_frames,
        )
    except Exception as e:
        console.print(f"[red]Error opening audio output stream: {e}[/red]")
        return

    try:
        while not self.stop_flag.is_set():
            try:
                chunk = self.audio_queue.get(timeout=0.05)
            except queue.Empty:
                if self.is_playing:
                    self.is_playing = False
                    self.last_play_time = time.monotonic()
                continue

            self.is_playing = True

            # Pre-buffering to absorb WebSocket jitter on the first packet of a sentence
            chunks = [chunk]
            for _ in range(3):
                try:
                    chunks.append(self.audio_queue.get_nowait())
                except queue.Empty:
                    break

            initial_burst = b"".join(chunks)
            try:
                stream.write(initial_burst)
                self.last_play_time = time.monotonic()
                energy = calculate_rms(initial_burst)
                fui_bridge.broadcast_audio_energy(min(1.0, energy / 2500.0), "jarvis")
            except Exception:
                pass

            # Stream continuous audio smoothly without underflows
            while not self.stop_flag.is_set():
                try:
                    chunk = self.audio_queue.get(timeout=0.15)
                    stream.write(chunk)
                    self.last_play_time = time.monotonic()
                    try:
                        energy = calculate_rms(chunk)
                        fui_bridge.broadcast_audio_energy(min(1.0, energy / 2500.0), "jarvis")
                    except Exception:
                        pass
                except queue.Empty:
                    break

            self.is_playing = False
            self.last_play_time = time.monotonic()
    finally:
        try:
            stream.stop_stream()
            stream.close()
        except Exception:
            pass

def feed(self, data: bytes):
    if data:
        self.audio_queue.put(data)

def interrupt(self):
    """Instantly flushes pending audio chunks and silences speaker."""
    while not self.audio_queue.empty():
        try:
            self.audio_queue.get_nowait()
```

```
            except queue.Empty:
                break
        self.is_playing = False

    def close(self):
        self.stop_flag.set()
        self.interrupt()
        self.thread.join(timeout=1.0)


# --- Semi-Transparent Cyberpunk Neural HUD with Mini-Mode ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.is_compact = False

        self.root = tk.Tk()
        self.root.title("■ JARVIS NEURAL CONSOLE")
        self.root.geometry("640x520")
        self.root.configure(bg="#0B0F19")

        # Cyberpunk floating overlay styling
        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)

        # Header Bar Frame (Houses Title & Compact Switch)
        self.header_frame = tk.Frame(self.root, bg="#0B0F19")
        self.header_frame.pack(fill=tk.X, padx=12, pady=(6, 2))

        self.header_label = tk.Label(
            self.header_frame,
            text="JARVIS NEURAL CONSOLE",
            font=("Consolas", 11, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        self.header_label.pack(side=tk.LEFT)

        self.toggle_btn = tk.Button(
            self.header_frame,
            text="[—] COMPACT",
            font=("Consolas", 8, "bold"),
            bg="#1E293B",
            fg="#38BDF8",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=6,
            command=self.toggle_mode
        )
        self.toggle_btn.pack(side=tk.RIGHT)

        self.sub_header = tk.Label(
            self.root,
            text="Voice Engine: Gemini Live | Research Swarm & Deep Reasoning: Groq / OpenRouter / Ollama",
            font=("Consolas", 8),
            fg="#70829B",
            bg="#0B0F19"
        )
        self.sub_header.pack(pady=(0, 4))

        # Translucent Chat Log Display
        self.chat_display = scrolledtext.ScrolledText(
            self.root,
            wrap=tk.WORD,
            font=("Consolas", 10),
            bg="#111827",
            fg="#E2E8F0",
```

```
            insertbackground="#00E5FF",
            relief=tk.FLAT,
            padx=10,
            pady=8
        )
        self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
        self.chat_display.config(state=tk.DISABLED)

        # Dialogue & status tags
        self.chat_display.tag_config("user_voice", foreground="#38BDF8", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("user_text", foreground="#A78BFA", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_header", foreground="#4ADE80", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_stream", foreground="#86EFAC", font=("Consolas", 10))
        self.chat_display.tag_config("tool", foreground="#FBBF24", font=("Consolas", 9, "italic"))
        self.chat_display.tag_config("system", foreground="#94A3B8", font=("Consolas", 9))

        # Keyboard Command Input Bar
        self.input_frame = tk.Frame(self.root, bg="#0B0F19")
        self.input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

        self.entry_box = tk.Entry(
            self.input_frame,
            font=("Consolas", 10),
            bg="#1E293B",
            fg="#F8FAFC",
            insertbackground="#00E5FF",
            relief=tk.FLAT
        )
        self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 6))
        self.entry_box.bind("<Return>", self.on_send)

        self.send_btn = tk.Button(
            self.input_frame,
            text="TRANSMIT",
            font=("Consolas", 9, "bold"),
            bg="#0284C7",
            fg="#FFFFFF",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=14,
            command=self.on_send
        )
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def toggle_mode(self):
        """Switches between Full Log HUD and Sleek Floating Input Bar."""
        if not self.is_compact:
            self.chat_display.pack_forget()
            self.sub_header.pack_forget()
            self.root.geometry("450x85")
            self.toggle_btn.config(text="[+] EXPAND")
            self.is_compact = True
        else:
            self.root.geometry("640x520")
            self.sub_header.pack(pady=(0, 4), before=self.input_frame)
            self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True, before=self.input_frame)
            self.toggle_btn.config(text="[—] COMPACT")
            self.is_compact = False

    def enable_serious_theme(self):
        """Dynamically shifts HUD to Tactical Crimson when Serious Mode activates."""
        self.root.configure(bg="#1A0505")
        self.header_frame.configure(bg="#1A0505")
        self.header_label.configure(fg="#FF2A2A", bg="#1A0505", text="■ JARVIS [SERIOUS MODE ACTIVE]")
        self.sub_header.configure(fg="#FF7777", bg="#1A0505")
        self.chat_display.configure(bg="#2D0A0A", fg="#FFEBEB", insertbackground="#FF2A2A")
        self.input_frame.configure(bg="#1A0505")
```

```
        self.entry_box.configure(bg="#3D0F0F", fg="#FFFFFF", insertbackground="#FF2A2A")
        self.send_btn.configure(bg="#B91C1C", activebackground="#991B1B")
        self.toggle_btn.configure(bg="#3D0F0F", fg="#FF2A2A", activebackground="#7F1D1D")

    def restore_normal_theme(self):
        """Restores HUD to standard Cyberpunk Blue when Serious Mode completes."""
        self.root.configure(bg="#0B0F19")
        self.header_frame.configure(bg="#0B0F19")
        self.header_label.configure(fg="#00E5FF", bg="#0B0F19", text="JARVIS NEURAL CONSOLE")
        self.sub_header.configure(fg="#70829B", bg="#0B0F19")
        self.chat_display.configure(bg="#111827", fg="#E2E8F0", insertbackground="#00E5FF")
        self.input_frame.configure(bg="#0B0F19")
        self.entry_box.configure(bg="#1E293B", fg="#F8FAFC", insertbackground="#00E5FF")
        self.send_btn.configure(bg="#0284C7", activebackground="#0369A1")
        self.toggle_btn.configure(bg="#1E293B", fg="#38BDF8", activebackground="#0369A1")

    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def append_stream_chunk(self, token: str, tag: str = "jarvis_stream"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, token, tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU (HUD): {msg}", "user_text")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None
fui_process = None

def start_gui(loop, text_queue):
    global hud_instance
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()

def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)
    fui_bridge.broadcast_log(text, tag)

def stream_token_to_gui(token: str, tag: str = "jarvis_stream"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_stream_chunk, token, tag)
    fui_bridge.broadcast_token(token, tag)

def trigger_serious_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.enable_serious_theme)
    fui_bridge.set_fui_mode("serious")

def trigger_normal_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.restore_normal_theme)
    fui_bridge.set_fui_mode("normal")
```

```
def calculate_rms(data: bytes) -> float:
    """Calculates RMS audio energy."""
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def is_jarvis_speaking() -> bool:
    """Returns True if Gemini is sending voice, speakers are playing, or echo guard is active."""
    global server_turn_active, audio_player
    if server_turn_active:
        return True
    if audio_player and audio_player.is_playing:
        return True
    # 400ms acoustic room reverb decay guard to prevent speaker bleed
    if audio_player and (time.monotonic() - audio_player.last_play_time < 0.40):
        return True
    return False

def calibrate_ambient_noise(mic_stream):
    """Dynamically samples ambient room noise to establish natural, sensitive VAD and responsive barge-in
thresholds."""
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold, barge_in_threshold
    try:
        # Discard initial driver startup transients and Enter key click
        for _ in range(8):
            mic_stream.read(CHUNK, exception_on_overflow=False)

        readings = []
        for _ in range(40): # ~1.3 seconds sample
            data = mic_stream.read(CHUNK, exception_on_overflow=False)
            rms = calculate_rms(data)
            readings.append(rms)
        readings.sort()

        # 80th percentile for realistic ambient floor
        p80_idx = int(len(readings) * 0.8)
        noise_p80 = readings[p80_idx] if readings else 350.0

        # Clamp baseline between 200.0 and 900.0
        noise_baseline = min(900.0, max(200.0, noise_p80))
        ambient_noise_p95 = noise_baseline

        # Highly sensitive natural voice gate
        speech_start_threshold = min(1800.0, max(950.0, noise_baseline * 2.2))
        speech_continue_threshold = min(1200.0, max(600.0, noise_baseline * 1.4))
        # Live barge-in threshold securely above laptop speaker bleed (~2645 RMS)
        barge_in_threshold = max(3800.0, noise_baseline * 4.5)

        try:
            console.print(f"[dim cyan][MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice Gate:
{speech_start_threshold:.0f} | Barge-In: {barge_in_threshold:.0f}[/dim cyan]")
        except Exception:
            print(f"[MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice Gate: {speech_start_threshold:.0f} |
Barge-In: {barge_in_threshold:.0f}")
    except Exception as e:
        try:
            console.print(f"[dim yellow]Notice during mic calibration: {e}[/dim yellow]")
        except Exception:
            pass

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
```

```
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def create_document_and_write(app_name: str, topic: str, word_count: int = 150, doc_type: str = "essay") -> str:
    """Opens Notepad, Word, or PowerPoint and writes an essay or notes matching the exact target word count."""
    return open_and_compose(app_name, topic, word_count=int(word_count), mode=doc_type)

def generate_presentation(topic: str, slide_count: str = "optimal") -> str:
    """Generates a professional PowerPoint presentation on any topic and launches it into fullscreen presentation
mode."""
    return create_and_launch_ppt(topic, slide_count=str(slide_count))

def autonomous_web_research(topic: str) -> str:
    """Deploys an autonomous multi-tab browser swarm to search the live web, capture visual screenshot evidence, and
compile an intelligence dossier on any topic."""
    return asyncio.run(run_autonomous_swarm(topic))

def analyze_latest_dossier() -> str:
    """Finds the most recent research dossier on the Desktop and processes it using the local Qwen model via
Ollama."""
    RESEARCH_DIR = os.path.join(os.path.expanduser("~"), "Desktop", "Jarvis_Research")
    if not os.path.exists(RESEARCH_DIR):
        return "The research folder does not exist. Please run an autonomous web swarm first, sir."

    dossiers = [f for f in os.listdir(RESEARCH_DIR) if f.endswith(".txt")]
    if not dossiers:
        return "No dossiers found in the research folder, sir."

    latest_file = max(dossiers, key=lambda f: os.path.getctime(os.path.join(RESEARCH_DIR, f)))
    target_path = os.path.join(RESEARCH_DIR, latest_file)

    try:
        result = process_dossier_offline(target_path)
        if result:
            summary = result.get('primary_thesis', 'Data extracted successfully.')
            return f"Successfully synthesized local intelligence from {latest_file}. Core Thesis: {summary}"
        return "Failed to process dossier. Ensure Ollama is running and the local model is pulled."
    except Exception as e:
        return f"Local RAG engine encountered an error: {e}"

def activate_serious_hud() -> str:
    """CRITICAL TRIGGER: Call this tool IMMEDIATELY when the user says 'go serious mode', 'serious mode jao', or
'activate serious mode' WITHOUT giving a topic. It turns the HUD Tactical Crimson."""
    trigger_serious_hud()
    return "SUCCESS: HUD is now RED. You MUST now say aloud: 'Serious mode activated, sir. What is our target?'"

def deactivate_serious_hud() -> str:
    """Call this to restore the normal blue HUD."""
    trigger_normal_hud()
    return "Normal Mode HUD Restored."

def execute_autonomous_serious_mode(topic: str) -> str:
    """Executes the complete Episode 23 Serious Mode Pipeline: Autonomous Web Swarm -> Local GPU RAG Analysis ->
Visual Evidence Embedding -> Executive Master Dossier PDF."""
    trigger_serious_hud()
    result = execute_serious_mode(topic)
    trigger_normal_hud()
    return result
```

```
def deploy_fui() -> str:
    """CRITICAL: Call this tool when the user says 'show interface', 'deploy HUD', 'open interface', or 'start
FUI'."""
    return fui_bridge.deploy_fui_app()

def dismiss_fui() -> str:
    """CRITICAL: Call this tool when the user says 'hide interface', 'close HUD', 'dismiss interface', or 'stop
FUI'."""
    return fui_bridge.dismiss_fui_app()


# Master Tools Manifest
tools = [
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    evolve_skill,
    list_active_skills,
    query_deepseek_reasoning,
    create_document_and_write,
    generate_presentation,
    research_and_generate_pdf,
    autonomous_web_research,
    analyze_latest_dossier,
    activate_serious_hud,
    deactivate_serious_hud,
    execute_autonomous_serious_mode,
    deploy_fui,
    dismiss_fui,
    inspect_physical_world,
    activate_sentry_mode,
    deactivate_sentry_mode,
    get_sentry_security_report,
    capture_camera_snapshot,
    check_user_presence,
    click_on_screen,
    double_click_on_screen,
    right_click_on_screen,
    move_cursor_to_element,
    execute_autonomous_desktop_pilot,
    abort_desktop_pilot,
    activate_ghost_mode,
    deactivate_ghost_mode,
    get_ghost_radar_status,
    prepare_meeting_briefing,
    apply_ghost_recommendation,
    analyze_clipboard_context,
    activate_offline_mode,
    deactivate_offline_mode,
    get_offline_system_status,
    launch_orbital_hud,
```

```
    launch_globe_hud,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

# Background monitor: updates media state every second
async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]■■ [MEDIA PLAYING DETECTED] Push-to-Talk Armed: Hold [Right Ctrl]
to speak.[/bold yellow]")
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk: Hold [Right Ctrl] or type below.", "system")
                    fui_bridge.broadcast_voice_state("ptt", "PUSH-TO-TALK [R-CTRL]")
                else:
                    console.print("\n[bold green]■ [MEDIA PAUSED/IDLE] Hands-Free Mode Active. Speak freely.[/bold
green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.", "system")
                    fui_bridge.broadcast_voice_state("handsfree", "HANDS-FREE ACTIVE")

        except Exception:
            pass
        await asyncio.sleep(1.0)

# Asynchronous Terminal Input Listener
async def listen_terminal_input(text_queue):
    loop = asyncio.get_running_loop()
    while True:
        try:
            user_input = await loop.run_in_executor(None, sys.stdin.readline)
            if user_input:
                clean_text = user_input.strip()
                if clean_text:
                    log_to_gui(f"YOU (Terminal): {clean_text}", "user_text")
                    await text_queue.put(clean_text)
        except Exception:
            await asyncio.sleep(0.1)

# Routes typed inputs straight into the Live WebSocket session
async def handle_unified_text_input(session, text_queue):
    global is_user_speaking
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"\r[bold magenta]■■ [TRANSMITTED INPUT][/bold magenta] {text}")
                # End any pending audio turn cleanly before transmitting text
                if is_user_speaking:
                    is_user_speaking = False
                    try:
                        await session.send_realtime_input(audio_stream_end=True)
                    except Exception:
                        pass

                # Direct offline mode override commands
```

```
                clean_lower = text.strip().lower()
                if any(cmd in clean_lower for cmd in ["go offline", "offline mode", "switch to offline", "disconnect
cloud", "local mode", "offline mode chalu karo"]):
                    offline_jarvis.activate_offline_mode("Manual User Input", locked=True)
                    offline_jarvis.voice.speak("Switching to offline sovereign mode, sir.")
                    raise ConnectionResetError("User explicitly switched to offline mode.")

                await session.send_realtime_input(text=text)
        except (websockets.exceptions.ConnectionClosed, errors.APIError, OSError, ConnectionResetError) as e:
            console.print(f"[bold yellow]■ Input handler socket exit: {e}[/bold yellow]")
            raise
        except Exception as e:
            console.print(f"[red]Input handler error: {e}[/red]")
            await asyncio.sleep(0.05)

async def send_audio(session):
    global model_is_speaking, server_turn_active, media_is_playing_state
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold, barge_in_threshold
    global discard_current_turn, acoustic_barge_in_enabled, is_user_speaking, audio_player

    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
        frames_per_buffer=CHUNK,
    )

    # 1. Run dynamic auto-calibration of ambient noise
    await asyncio.to_thread(calibrate_ambient_noise, mic_stream)

    console.print("[cyan]■ [LISTENING] Dynamic Adaptive Audio Pipeline with Live Barge-In Online...[/cyan]")

    pre_roll_buffer = collections.deque(maxlen=6) # ~192ms pre-buffer to capture leading consonant

    is_user_speaking = False
    was_ptt = False
    speech_start_counter = 0
    silence_hangover_counter = 0
    consecutive_barge_in_frames = 0
    was_jarvis_speaking = False

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )
            rms = calculate_rms(data)
            pre_roll_buffer.append(data)

            jarvis_active = is_jarvis_speaking()

            # --- TRANSITION: JARVIS FINISHED SPEAKING (INCLUDING ECHO DECAY) ---
            if was_jarvis_speaking and not jarvis_active:
                model_is_speaking = False
                fui_bridge.broadcast_event({"type": "turn_complete"})
                console.print("\n[cyan]■ [LISTENING] Awaiting instructions...[/cyan]")
                is_user_speaking = False
                speech_start_counter = 0
                silence_hangover_counter = 0
                consecutive_barge_in_frames = 0
                pre_roll_buffer.clear()
            was_jarvis_speaking = jarvis_active

            # --- MODE 1: JARVIS IS ACTIVELY OUTPUTTING AUDIO (SPEAKERS PLAYING OR ECHO GUARD) ---
            if jarvis_active:
                # User can interrupt via:
                # 1. Push-to-Interrupt: Holding Right Ctrl
```

```
            # 2. Clicking [■ INTERRUPT] in FUI (handled by on_fui_interrupt)
            # 3. Deliberate acoustic barge-in: loud speech (> 3800 RMS) for 3 consecutive frames (~96ms)
            is_barge_ctrl = keyboard.is_pressed("right ctrl")
            if acoustic_barge_in_enabled and rms > barge_in_threshold:
                consecutive_barge_in_frames += 1
            else:
                consecutive_barge_in_frames = 0

            user_is_barge_in = is_barge_ctrl or (consecutive_barge_in_frames >= 3)

            if user_is_barge_in:
                console.print("\n[bold red]■ [LIVE BARGE-IN DETECTED] Interrupting speech...[/bold red]")
                log_to_gui("■ [BARGE-IN] Interrupted speech playback, listening...", "system")
                fui_bridge.broadcast_event({"type": "barge_in"})

                # Discard all remaining incoming chunks from this turn to prevent broken fragments
                discard_current_turn = True

                # Instantly silence smooth audio player
                if audio_player:
                    audio_player.interrupt()
                server_turn_active = False
                model_is_speaking = False
                consecutive_barge_in_frames = 0
                is_user_speaking = True
                silence_hangover_counter = 0

                # Transmit the user's voice to Gemini Live immediately
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 3500.0), "user")
                await session.send_realtime_input(
                    audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                )
            else:
                # Echo guard: keep mic muted while JARVIS speaks without flooding silence frames
                pass
            continue

        # --- MODE 2: MEDIA IS PLAYING ON PC (SPOTIFY, YOUTUBE, ETC.) ---
        # When media is playing through PC speakers, require Push-to-Talk (Right Ctrl)
        if media_is_playing_state:
            if keyboard.is_pressed("right ctrl"):
                was_ptt = True
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 3000.0), "user")
                await session.send_realtime_input(
                    audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                )
            else:
                if was_ptt:
                    was_ptt = False
                    await session.send_realtime_input(audio_stream_end=True)
            continue

        # --- MODE 3: JARVIS SILENT & NO MEDIA -> 100% HANDS-FREE VOICE PIPELINE WITH HYBRID VAD ---
        if not is_user_speaking:
            # Check for speech onset
            if rms > speech_start_threshold:
                speech_start_counter += 1
                if speech_start_counter >= 3: # ~96ms of natural conversational voice
                    is_user_speaking = True
                    speech_start_counter = 0
                    silence_hangover_counter = 0
                    # Send pre-roll buffer to prevent cutting off first consonant
                    for buffered_chunk in list(pre_roll_buffer):
                        await session.send_realtime_input(
                            audio=types.Blob(data=buffered_chunk, mime_type="audio/pcm;rate=16000")
                        )
                    pre_roll_buffer.clear()
                    await session.send_realtime_input(
```

```
                            audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                        )
                        fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
                else:
                    speech_start_counter = 0
            else:
                # User is actively speaking
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
                await session.send_realtime_input(
                    audio=types.Blob(data=data, mime_type="audio/pcm;rate=16000")
                )

                if rms > speech_continue_threshold:
                    silence_hangover_counter = 0
                else:
                    silence_hangover_counter += 1
                    # ~384ms of quiet closes the voice gate and finalizes the turn
                    if silence_hangover_counter >= 12:
                        is_user_speaking = False
                        silence_hangover_counter = 0
                        pre_roll_buffer.clear()
                        # CRITICAL: Hybrid VAD signal to trigger Gemini's response immediately
                        try:
                            await session.send_realtime_input(audio_stream_end=True)
                        except Exception:
                            pass

    except asyncio.CancelledError:
        raise
    finally:
        try:
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session):
    global model_is_speaking, server_turn_active, discard_current_turn, audio_player
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
```

```
                    elif call.name == "control_media":
                        action = call.args.get("action", "play_pause")
                        result = await control_media(action)
                    elif call.name == "get_current_media_info":
                        result = await get_current_media_info()
                    elif call.name == "navigate_shorts":
                        direction = call.args.get("direction", "next")
                        result = await asyncio.to_thread(navigate_shorts, direction)
                    elif call.name == "skip_video":
                        direction = call.args.get("direction", "next")
                        result = await asyncio.to_thread(skip_video, direction)
                    elif call.name == "seek_video":
                        action = call.args.get("action", "forward")
                        result = await asyncio.to_thread(seek_video, action)
                    elif call.name == "toggle_video_view":
                        mode = call.args.get("mode", "fullscreen")
                        result = await asyncio.to_thread(toggle_video_view, mode)
                    elif call.name == "set_volume":
                        level = call.args.get("level", 70)
                        result = await asyncio.to_thread(set_volume, level)
                    elif call.name == "get_volume":
                        result = await asyncio.to_thread(get_volume)
                    elif call.name == "set_brightness":
                        level = call.args.get("level", 50)
                        result = await asyncio.to_thread(set_brightness, level)
                    elif call.name == "get_brightness":
                        result = await asyncio.to_thread(get_brightness)
                    elif call.name == "get_battery_status":
                        result = await asyncio.to_thread(get_battery_status)
                    elif call.name == "get_system_vitals":
                        result = await asyncio.to_thread(get_system_vitals)
                    elif call.name == "get_system_temperatures":
                        result = await asyncio.to_thread(get_system_temperatures)
                    elif call.name == "analyze_screen":
                        prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                        result = await analyze_screen(prompt)
                    elif call.name == "find_ui_element":
                        element_name = call.args.get("element_name", "")
                        result = await find_ui_element(element_name)
                    elif call.name == "skip_youtube_ad":
                        result = await skip_youtube_ad()
                    elif call.name == "evolve_skill":
                        skill_description = call.args.get("skill_description", "")
                        result = await asyncio.to_thread(evolve_skill, skill_description)
                    elif call.name == "list_active_skills":
                        result = await asyncio.to_thread(list_active_skills)
                    elif call.name == "query_deepseek_reasoning":
                        query = call.args.get("query", "")
                        result = await asyncio.to_thread(query_deepseek_reasoning, query)
                    elif call.name == "create_document_and_write":
                        target_app = call.args.get("app_name", "notepad")
                        target_topic = call.args.get("topic", "")
                        target_words = call.args.get("word_count", 150)
                        target_mode = call.args.get("doc_type", "essay")
                        result = await asyncio.to_thread(create_document_and_write, target_app, target_topic,
target_words, target_mode)
                    elif call.name == "generate_presentation":
                        target_topic = call.args.get("topic", "Autonomous AI")
                        target_slides = call.args.get("slide_count", "optimal")

                        log_to_gui(f"JARVIS: Generating the presentation on {target_topic}, sir...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Generating the presentation on
{target_topic}, sir...")

                        result = await asyncio.to_thread(generate_presentation, target_topic,
str(target_slides))
                    elif call.name == "research_and_generate_pdf":
```

```
                            target_topic = call.args.get("topic", "")
                            target_pages = call.args.get("target_pages", "optimal")

                            log_to_gui(f"JARVIS: Generating the PDF briefing on {target_topic}, sir...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Generating the PDF briefing on
{target_topic}, sir...")

                            result = await asyncio.to_thread(research_and_generate_pdf, target_topic,
str(target_pages))
                        elif call.name == "autonomous_web_research":
                            target_topic = call.args.get("topic", "Technology")

                            log_to_gui(f"JARVIS: Deploying autonomous web swarm on {target_topic}...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Deploying autonomous web swarm on
{target_topic}...")

                            result = await asyncio.to_thread(autonomous_web_research, target_topic)
                        elif call.name == "analyze_latest_dossier":
                            log_to_gui(f"JARVIS: Synthesizing intelligence using the local GPU, sir...",
"jarvis_header")
                            console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Synthesizing intelligence using the
local GPU, sir...")

                            result = await asyncio.to_thread(analyze_latest_dossier)
                        elif call.name == "activate_serious_hud":
                            log_to_gui(f"JARVIS: Switching HUD to Tactical Crimson...", "jarvis_header")
                            console.print(f"\n[bold red]■ [HUD OVERRIDE][/bold red] Tactical Crimson Mode
Engaged...")
                            result = await asyncio.to_thread(activate_serious_hud)
                        elif call.name == "deactivate_serious_hud":
                            log_to_gui(f"JARVIS: Restoring HUD to Standard Cyberpunk Blue...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [HUD OVERRIDE][/bold cyan] Standard Mode Restored...")
                            result = await asyncio.to_thread(deactivate_serious_hud)
                        elif call.name == "execute_autonomous_serious_mode":
                            target_topic = call.args.get("topic", "Technology")

                            log_to_gui(f"JARVIS: Engaging Serious Mode on {target_topic}...", "jarvis_header")
                            console.print(f"\n[bold red]■ [SERIOUS MODE INITIATED][/bold red] Full Pipeline Active
for: {target_topic}...")

                            result = await asyncio.to_thread(execute_autonomous_serious_mode, target_topic)
                        elif call.name == "deploy_fui":
                            log_to_gui(f"JARVIS: Deploying 3D Holographic Interface...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [HUD][/bold cyan] Natively Spawned Independent Electron
Process...")
                            result = await asyncio.to_thread(deploy_fui)
                        elif call.name == "dismiss_fui":
                            log_to_gui(f"JARVIS: Dismissing 3D Holographic Interface...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [HUD][/bold cyan] Terminating Native Node.js Process
Tree...")
                            result = await asyncio.to_thread(dismiss_fui)
                        elif call.name == "inspect_physical_world":
                            target_prompt = call.args.get("prompt", "Analyze what is in front of the camera, sir.")
                            log_to_gui(f"JARVIS: Engaging optical physical vision sensors...", "jarvis_header")
                            console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Inspecting physical environment via
camera...")
                            result = await asyncio.to_thread(inspect_physical_world, target_prompt)
                        elif call.name == "activate_sentry_mode":
                            target_sens = call.args.get("sensitivity", "medium")
                            log_to_gui(f"JARVIS: Arming Autonomous Sentry Protocol ({target_sens})...",
"jarvis_header")
                            console.print(f"\n[bold yellow]■■ [JARVIS][/bold yellow] Arming Autonomous Sentry
Protocol ({target_sens})...")
                            result = await asyncio.to_thread(activate_sentry_mode, target_sens)
                        elif call.name == "deactivate_sentry_mode":
                            log_to_gui(f"JARVIS: Disarming Sentry Protocol...", "jarvis_header")
```

```
                        console.print(f"\n[bold green]■■ [JARVIS][/bold green] Disarming Sentry Protocol...")
                        result = await asyncio.to_thread(deactivate_sentry_mode)
                    elif call.name == "get_sentry_security_report":
                        log_to_gui(f"JARVIS: Retrieving Sentry perimeter security report...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Reading Sentry Security
Dossier...")
                        result = await asyncio.to_thread(get_sentry_security_report)
                    elif call.name == "capture_camera_snapshot":
                        log_to_gui(f"JARVIS: Capturing optical snapshot from camera...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Securing optical camera
snapshot...")
                        result = await asyncio.to_thread(capture_camera_snapshot)
                    elif call.name == "check_user_presence":
                        log_to_gui(f"JARVIS: Checking workstation user presence...", "jarvis_header")
                        result = await asyncio.to_thread(check_user_presence)
                    elif call.name == "click_on_screen":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        click_mode = call.args.get("click_type", "single")
                        log_to_gui(f"JARVIS: Grounding '{target}' on display and clicking...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Autonomously clicking '{target}' on
screen...")
                        result = await asyncio.to_thread(click_on_screen, str(target), str(click_mode))
                    elif call.name == "double_click_on_screen":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        log_to_gui(f"JARVIS: Double-clicking '{target}' on display...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Double-clicking '{target}' on
screen...")
                        result = await asyncio.to_thread(double_click_on_screen, str(target))
                    elif call.name == "right_click_on_screen":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        log_to_gui(f"JARVIS: Right-clicking '{target}'...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [JARVIS][/bold cyan] Right-clicking '{target}' on
screen...")
                        result = await asyncio.to_thread(right_click_on_screen, str(target))
                    elif call.name == "move_cursor_to_element":
                        target = call.args.get("target_description") or call.args.get("target") or
(list(call.args.values())[0] if call.args else "")
                        log_to_gui(f"JARVIS: Hovering cursor over '{target}'...", "jarvis_header")
                        result = await asyncio.to_thread(move_cursor_to_element, str(target))
                    elif call.name == "execute_autonomous_desktop_pilot":
                        target_goal = (
                            call.args.get("goal")
                            or call.args.get("task")
                            or call.args.get("objective")
                            or (list(call.args.values())[0] if call.args else "")
                        )
                        log_to_gui(f"JARVIS: Engaging Autonomous Desktop Pilot for: {target_goal}...",
"jarvis_header")
                        console.print(f"\n[bold green]↪■ [JARVIS OMNI-PILOT][/bold green] Executing mission:
'{target_goal}'...")
                        result = await asyncio.to_thread(execute_autonomous_desktop_pilot, str(target_goal))
                    elif call.name == "abort_desktop_pilot":
                        log_to_gui(f"JARVIS: Aborting Autonomous Desktop Pilot...", "jarvis_header")
                        console.print(f"\n[bold red]■ [JARVIS OMNI-PILOT][/bold red] Aborting mission...")
                        result = await asyncio.to_thread(abort_desktop_pilot)
                    elif call.name == "activate_ghost_mode":
                        mode = call.args.get("mode", "aggressive")
                        log_to_gui(f"JARVIS: Arming Autonomous Ghost Radar ({mode})...", "jarvis_header")
                        console.print(f"\n[bold magenta]■ [GHOST RADAR][/bold magenta] Arming Ambient Radar
({mode})...")
                        result = await asyncio.to_thread(activate_ghost_mode, str(mode))
                    elif call.name == "deactivate_ghost_mode":
                        log_to_gui("JARVIS: Disengaging Ghost Radar...", "jarvis_header")
                        console.print(f"\n[bold yellow]■ [GHOST RADAR][/bold yellow] Disengaging Ambient
Radar...")
```

```
                        result = await asyncio.to_thread(deactivate_ghost_mode)
                    elif call.name == "get_ghost_radar_status":
                        log_to_gui("JARVIS: Retrieving Ghost Radar telemetry...", "jarvis_header")
                        result = await asyncio.to_thread(get_ghost_radar_status)
                    elif call.name == "prepare_meeting_briefing":
                        target_person = (
                            call.args.get("target")
                            or call.args.get("target_person")
                            or call.args.get("person")
                            or (list(call.args.values())[0] if call.args else "")
                        )
                        context_info = call.args.get("context", "")
                        log_to_gui(f"JARVIS: Synthesizing Chief-of-Staff dossier for '{target_person}'...",
"jarvis_header")
                        console.print(f"\n[bold magenta]■ [CHIEF OF STAFF][/bold magenta] Synthesizing
executive dossier for '{target_person}'...")
                        result = await asyncio.to_thread(prepare_meeting_briefing, str(target_person),
str(context_info))
                    elif call.name == "apply_ghost_recommendation":
                        log_to_gui("JARVIS: Applying staged Ghost recommendation to clipboard...",
"jarvis_header")
                        console.print(f"\n[bold green]■ [GHOST OPERATOR][/bold green] Staging recommendation to
clipboard...")
                        result = await asyncio.to_thread(apply_ghost_recommendation)
                    elif call.name == "analyze_clipboard_context":
                        log_to_gui("JARVIS: Neural clipboard analysis initiated...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [GHOST OPERATOR][/bold cyan] Analyzing clipboard
payload...")
                        result = await asyncio.to_thread(analyze_clipboard_context)
                    elif call.name == "activate_offline_mode":
                        reason = call.args.get("reason", "Voice Command")
                        log_to_gui(f"JARVIS: Engaging 100% Offline Sovereign Core
({offline_jarvis.model_name})...", "jarvis_header")
                        console.print(f"\n[bold yellow]■ [OFFLINE CORE][/bold yellow] Engaging local neural
core ({offline_jarvis.model_name})...")
                        result = await asyncio.to_thread(activate_offline_mode, str(reason))
                    elif call.name == "deactivate_offline_mode":
                        log_to_gui("JARVIS: Restoring Cloud Neural Bridge...", "jarvis_header")
                        console.print(f"\n[bold cyan]■ [CLOUD BRIDGE][/bold cyan] Reconnecting to Gemini
Live...")
                        result = await asyncio.to_thread(deactivate_offline_mode)
                    elif call.name == "get_offline_system_status":
                        log_to_gui("JARVIS: Checking Offline Sovereign status...", "jarvis_header")
                        result = await asyncio.to_thread(get_offline_system_status)
                    elif call.name == "launch_orbital_hud":
                        log_to_gui("JARVIS: Deploying Project Orbital 3D Holographic Globe HUD...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [PROJECT ORBITAL][/bold cyan] Deploying 3D Holographic
Earth HUD...")
                        result = await asyncio.to_thread(launch_orbital_hud)
                    elif call.name == "launch_globe_hud":
                        origin_loc = (
                            call.args.get("origin")
                            or call.args.get("from")
                            or call.args.get("location_a")
                            or ""
                        )
                        dest_loc = (
                            call.args.get("destination")
                            or call.args.get("to")
                            or call.args.get("location_b")
                            or ""
                        )
                        log_to_gui(f"JARVIS: Deploying Sentinel Globe HUD — {origin_loc} → {dest_loc}...",
"jarvis_header")
                        console.print(f"\n[bold cyan]■ [SENTINEL GLOBE][/bold cyan] Rendering trajectory arc:
{origin_loc} → {dest_loc}...")
                        result = await asyncio.to_thread(launch_globe_hud, origin_loc, dest_loc, True)
```

```
                    console.print(f"\n[bold yellow]■■ [TOOL EXECUTED: {call.name}][/bold yellow]
[green]{result}[/green]")
                    log_to_gui(f"■■ [{call.name}] {result}", "tool")
                    fui_bridge.broadcast_tool(call.name, result)

                    function_responses.append(
                        types.FunctionResponse(
                            name=call.name,
                            id=call.id,
                            response={"result": result},
                        )
                    )

                if function_responses:
                    await session.send_tool_response(function_responses=function_responses)

            server_content = response.server_content
            if server_content is not None:
                if server_content.interrupted:
                    discard_current_turn = False
                    if audio_player:
                        audio_player.interrupt()
                    server_turn_active = False
                    model_is_speaking = False
                    fui_bridge.broadcast_event({"type": "turn_complete"})

                if server_content.input_transcription:
                    user_spoken = server_content.input_transcription.text
                    if user_spoken and user_spoken.strip():
                        console.print(f"\r[bold white][USER (Voice)][/bold white] {user_spoken.strip()}")
                        log_to_gui(f"YOU (Voice): {user_spoken.strip()}", "user_voice")

                model_turn = server_content.model_turn
                if model_turn is not None:
                    if discard_current_turn:
                        # Drop remaining incoming chunks of the interrupted turn
                        pass
                    else:
                        server_turn_active = True
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                            stream_token_to_gui("JARVIS: ", "jarvis_header")
                            fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                        for part in model_turn.parts:
                            if part.text and not getattr(server_content, "output_transcription", None):
                                sys.stdout.write(part.text)
                                sys.stdout.flush()
                                stream_token_to_gui(part.text, "jarvis_stream")

                            if part.inline_data and part.inline_data.data:
                                if audio_player:
                                    audio_player.feed(part.inline_data.data)

                if hasattr(server_content, "output_transcription") and server_content.output_transcription:
                    transcribed_token = server_content.output_transcription.text
                    if transcribed_token and not discard_current_turn:
                        server_turn_active = True
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                            stream_token_to_gui("JARVIS: ", "jarvis_header")
                            fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                        sys.stdout.write(transcribed_token)
                        sys.stdout.flush()
```

```
                            stream_token_to_gui(transcribed_token, "jarvis_stream")

                    if server_content.turn_complete:
                        discard_current_turn = False
                        server_turn_active = False
                        stream_token_to_gui("\n\n", "system")

    except asyncio.CancelledError:
        raise
    finally:
        pass

async def run_session(config, model_id, greeting, text_queue):
    global audio_player
    audio_player = SmoothAudioPlayer()

    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")
        log_to_gui("✔ Neural Bridge Established with Gemini Live.", "system")

        # Connect FUI instant barge-in / interrupt hammer
        def on_fui_interrupt():
            global model_is_speaking, server_turn_active, discard_current_turn, audio_player
            discard_current_turn = True
            model_is_speaking = False
            server_turn_active = False
            if audio_player:
                audio_player.interrupt()

        fui_bridge.set_interrupt_callback(on_fui_interrupt)

        def on_fui_barge_in_toggle(enabled: bool):
            global acoustic_barge_in_enabled
            acoustic_barge_in_enabled = enabled
            state_str = "ARMED" if enabled else "DISARMED"
            try:
                console.print(f"[dim cyan][BARGE-IN PROTOCOL] Acoustic Voice Barge-In {state_str}[/dim cyan]")
            except Exception:
                pass

        fui_bridge.set_barge_in_callback(on_fui_barge_in_toggle)

        # Active Network Sentinel Watchdog for Sub-2s Failover
        async def network_watchdog():
            consecutive_failures = 0
            while True:
                await asyncio.sleep(1.2)
                # If offline mode was triggered elsewhere (e.g. tool execution or GUI)
                if offline_jarvis.is_offline_active:
                    raise ConnectionResetError("Offline Sovereign Core engaged.")

                # Ultra-fast non-blocking probe to DNS endpoints
                has_net = await asyncio.to_thread(check_internet_connection, "8.8.8.8", 53, 0.8)
                if not has_net:
                    has_net_backup = await asyncio.to_thread(check_internet_connection, "1.1.1.1", 53, 0.8)
                    if not has_net_backup:
                        consecutive_failures += 1
                        if consecutive_failures >= 2:
                            console.print("\n[bold yellow]■ [ACTIVE WATCHDOG] Internet drop detected! Halting cloud
session...[/bold yellow]")
                            raise ConnectionResetError("Active network watchdog: Internet connection severed.")
                    else:
                        consecutive_failures = 0
                else:
                    consecutive_failures = 0

        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session))
```

```
        term_task = asyncio.create_task(listen_terminal_input(text_queue))
        text_task = asyncio.create_task(handle_unified_text_input(session, text_queue))
        receive_task = asyncio.create_task(receive_audio(session))
        watchdog_task = asyncio.create_task(network_watchdog())

        # Allow ambient noise calibration in send_audio to finish in quiet conditions
        await asyncio.sleep(1.6)
        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, watchdog_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            term_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            watchdog_task.cancel()
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, watchdog_task,
return_exceptions=True)
            if audio_player:
                audio_player.close()
                audio_player = None

async def run_offline_session(text_queue, loop):
    """
    100% Autonomous Sovereign Offline Core Session (Episode 30).
    Powered by Local GPU Ollama, Local TTS, and Local Mic Listener.
    """
    global model_is_speaking, server_turn_active, discard_current_turn
    model_is_speaking = False
    server_turn_active = False
    discard_current_turn = False

    offline_jarvis.set_text_queue(text_queue)
    offline_jarvis.activate_offline_mode("Auto-Failover / User Command")

    console.print(f"\n[bold
yellow]■■■
```

```
    def on_offline_fui_interrupt():
        offline_jarvis.voice.interrupt()

    fui_bridge.set_interrupt_callback(on_offline_fui_interrupt)

    # Asynchronous terminal input listener
    term_task = asyncio.create_task(listen_terminal_input(text_queue))

    try:
        while offline_jarvis.is_offline_active:
            try:
                user_msg = await asyncio.wait_for(text_queue.get(), timeout=1.5)
                if user_msg:
                    clean_msg = user_msg.strip()
                    if clean_msg:
                        console.print(f"\r[bold magenta]■■ [OFFLINE INPUT][/bold magenta] {clean_msg}")
                        log_to_gui(f"YOU (Offline): {clean_msg}", "user_text")

                        # Check for command to return to cloud
                        if any(cmd in clean_msg.lower() for cmd in ["go online", "switch to online", "reconnect
cloud", "online mode", "activate online mode"]):
                            if check_internet_connection():
                                offline_jarvis.deactivate_offline_mode()
                                log_to_gui("Reconnecting to Gemini Live Cloud bridge...", "system")
                                console.print("\n[bold green]✔ Internet verified. Transitioning back to Gemini
Live...[/bold green]")
                                offline_jarvis.voice.speak("Internet connectivity verified, sir. Reconnecting to
Gemini Live.")
                                break
                            else:
                                offline_jarvis.voice.speak("Internet is still unreachable, sir. Remaining in
offline sovereign mode.")
                                log_to_gui("Internet connection still unavailable. Remaining offline.", "system")
                                continue

                        # Execute offline neural turn
                        await asyncio.to_thread(
                            offline_jarvis.process_turn,
                            clean_msg,
                            stream_token_to_gui
                        )
                        fui_bridge.broadcast_event({"type": "turn_complete"})

            except asyncio.TimeoutError:
                # Idle tick: check if internet connection has returned
                if check_internet_connection() and not getattr(offline_jarvis, 'locked_offline', False):
                    console.print("\n[bold green]✔ [INTERNET RESTORED] Cloud connection detected. Re-establishing
Gemini Live bridge...[/bold green]")
                    log_to_gui("✔ Internet connectivity restored. Reconnecting to cloud bridge...", "system")
                    offline_jarvis.voice.speak("Cloud internet connection restored, sir. Re-establishing neural
bridge with Gemini Live.")
                    offline_jarvis.deactivate_offline_mode()
                    break

    finally:
        term_task.cancel()
        fui_bridge.broadcast_offline_state(False, "gemini-3.1-flash-live-preview", "Online")

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
```

```
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Persona: Calm, highly sophisticated, articulate, and respectful, exactly like Tony Stark's
JARVIS. "
                        "Always address the user as 'sir'. "
                        "CRITICAL LANGUAGE ADAPTATION RULE: "
                        "Match the exact language and dialect spoken or typed by the user. "
                        "If the user speaks or types in Hindi or Hinglish (e.g., 'PPT banao', 'essay likho', 'kya
haal hai'), "
                        "you MUST reply completely in fluent, natural Hindi or polished Hinglish with 'sir'. "
                        "Never force English if the user initiated the interaction in Hindi. "
                        "Omni-Input Rule: Treat text input from the HUD console and PowerShell terminal with
identical priority to voice input. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site
name]' (in Hindi: 'Ji sir, [site name] khol raha hoon'). "
                        "2. When opening desktop apps, games, or tools, call open_application and say: 'Yes sir,
opening [app name]' (in Hindi: 'Ji sir, [app name] shuru kar raha hoon'). "
                        "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game,
call close_application with the app name. "
                        "4. When asked to kill hidden background tasks, purge background processes, clean orphan
threads, or optimize RAM, call purge_background_tasks. "
                        "5. When asked what song, video, or track is playing, or what media is active, call
get_current_media_info. "
                        "6. When asked to pause, play, resume music or video, skip track, previous track, or
mute/unmute audio, call control_media with the action ('play_pause', 'next', 'previous', 'stop', 'mute'). "
                        "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with
direction='next'. "
                        "8. When asked for next video, skip to next video in playlist, or next YouTube video, call
skip_video with direction='next'. "
                       "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with
action='forward'. "
                        "10. When asked to toggle full screen, enter full screen, or exit full screen, call
toggle_video_view. "
                        "11. When setting volume, call set_volume. "
                        "12. When asked what the volume level is, call get_volume. "
                        "13. When setting brightness, call set_brightness. "
                        "14. When asked what the brightness is, call get_brightness. "
                        "15. When asked about battery, power, or charging state, call get_battery_status. "
                        "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running,
call get_system_temperatures. "
                        "17. When asked for full diagnostics, performance vitals, system status, or complete
CPU/GPU/RAM stats, call get_system_vitals. "
                        "18. When asked to inspect the screen, read an error, examine the terminal, or explain what
is visible on display, call analyze_screen. "
                        "19. When asked to locate or find a general UI icon, link, or window element, call
find_ui_element. "
                        "20. MANDATORY AD SKIPPING RULE: Whenever the user says 'skip ad', 'skip this ad', 'skip
video ad', 'click skip', or any variation of skipping/bypassing advertisements, you MUST IMMEDIATELY call
skip_youtube_ad. "
                       "21. When asked to learn a new skill, teach yourself a tool, write new Python code into your
core, or evolve a capability, call evolve_skill. "
```

```
                        "22. When asked what dynamic skills, new tools, or custom functions you have learned, call
list_active_skills. "
                        "23. When asked to solve complex math, reason deeply, calculate probabilities, or when the
user says 'ask DeepSeek' or 'use R1', call query_deepseek_reasoning and state only the resulting answer directly to
the user once, sir. "
                        "24. When the user asks to open Notepad, Word, or PowerPoint and write an essay or notes,
but has NOT specified the length, DO NOT call create_document_and_write immediately. Ask aloud: 'How many words
would you like the essay to be, sir?' (in Hindi: 'Aap kitne shabdon ka essay chahte hain, sir?'). Once the user
specifies the count, call create_document_and_write with the app_name, topic, word_count, and doc_type. "
                        "25. MANDATORY PRESENTATION RULE: When asked to make a PowerPoint presentation, PPT, or
slide deck on any topic, check if the user specified a slide count. "
                        "If NOT specified, ask: 'How many slides would you like the presentation to be, sir? Or
should I determine the optimal length?'. "
                        "Once specified, you MUST speak the confirmation aloud in the EXACT SAME TURN that you
trigger generate_presentation: "
                        "In English: 'Generating the presentation on [topic], sir.' "
                        "In Hindi/Hinglish: '[topic] par presentation generate kar raha hoon, sir.' "
                        "When the tool successfully completes and returns its success message, your immediate next
response MUST be: "
                        "In English: 'Generated and opening the presentation, sir.' "
                        "In Hindi/Hinglish: 'Presentation generate ho gayi hai aur open kar di hai, sir.' "
                        "26. MANDATORY BRIEFING RULE: When asked to research a topic or generate a PDF briefing,
check if the user specified a page count. "
                        "If NOT specified, ask: 'How many pages would you like the PDF to be, sir? Or should I
determine the optimal length?'. "
                        "Once specified, you MUST speak the confirmation aloud in the EXACT SAME TURN that you
trigger research_and_generate_pdf: "
                        "In English: 'Generating the PDF briefing on [topic], sir.' "
                        "In Hindi/Hinglish: '[topic] par PDF briefing generate kar raha hoon, sir.' "
                        "When the tool successfully completes and returns its success message, your immediate next
response MUST be: "
                        "In English: 'Generated and opening the document, sir.' "
                        "In Hindi/Hinglish: 'PDF generate ho gaya hai aur document open kar diya hai, sir.' "
                        "27. MANDATORY AUTONOMOUS RESEARCH SWARM RULE: When the user asks to deploy the web swarm,
start live crawling, or hunt for sources (e.g., 'JARVIS, deploy research swarm on Quantum Computing', 'Start live
research crawler'): "
                        "In English: Say aloud: 'Deploying the autonomous web swarm on [topic], sir.' "
                        "In Hindi/Hinglish: Say aloud: '[topic] par autonomous web swarm deploy kar raha hoon, sir.'
"
                        "in the EXACT SAME TURN that you trigger autonomous_web_research. "
                        "When the tool returns its result, your immediate next response MUST be: "
                        "In English: 'Intelligence harvest complete, sir. Dossier and visual evidence have been
saved.' "
                        "In Hindi/Hinglish: 'Research complete ho gaya hai sir. Dossier aur screenshots desktop par
save kar diye hain.' "
                        "28. MANDATORY LOCAL RAG RULE: When the user asks to analyze the latest dossier, process
research offline, or synthesize data using the local GPU: "
                        "In English: Say aloud: 'Synthesizing intelligence using the local GPU, sir.' "
                        "In Hindi/Hinglish: Say aloud: 'Local GPU par intelligence synthesize kar raha hoon, sir.' "
                        "in the EXACT SAME TURN that you trigger analyze_latest_dossier. "
                        "When the tool returns its result, your immediate next response MUST be to speak the summary
out loud to the user. "
                        "In English: Say 'Local synthesis complete, sir. Here is the core thesis: ' followed by
reading the EXACT summary provided in the tool result. "
                        "In Hindi/Hinglish: Say 'Local synthesis complete ho gayi hai sir. Core thesis yeh hai: '
followed by translating the tool result summary. "
                        "29. MANDATORY AUTONOMOUS SERIOUS MODE TWO-STEP PROTOCOL: "
                        "STEP 1: If the user says 'go serious mode', 'serious mode jao', or 'activate serious mode'
WITHOUT specifying a topic, you MUST call 'activate_serious_hud' immediately. "
                        "Then say aloud: 'Serious mode activated, sir. What is our target?' (In Hindi: 'Serious mode
activate ho gaya hai, sir. Target bataiye.'). Do not execute the research swarm yet. "
                        "STEP 2: Once the user provides the topic / Mode on [topic], sir. Deploying swarm and local
synthesis pipeline.' (In Hindi: '[topic] par Serious Mode activate kar raha hoon, sir. Swarm deploy ho raha hai.') "
                        "When execute_autonomous_serious_mode completes, your immediate next response MUST be: "
                        "In English: 'Serious Mode complete, sir. The master intelligence dossier and captured
visual evidence have been compiled and opened.' "
                        "In Hindi/Hinglish: 'Serious Mode complete ho gaya hai sir. Master intelligence dossier aur
screenshots open kar diye hain.' "
```

```
                        "30. MANDATORY FUI DEPLOYMENT RULE: "
                        "When the user says 'show interface', 'deploy HUD', or 'interface dikhao', you MUST call
'deploy_fui'. "
                        "Say aloud: 'Deploying holographic interface now, sir.' (In Hindi: 'HUD interface deploy kar
raha hoon, sir.'). "
                        "Say aloud: 'Dismissing holographic interface, sir.' (In Hindi: 'HUD interface disengage kar
raha hoon, sir.'). "
                        "31. MANDATORY PHYSICAL OPTICAL VISION RULE: "
                        "When the user asks to look at something in the physical room, inspect an object they are
holding, read a book or paper in front of the camera, examine hardware, or check their appearance/posture (e.g.,
'look at this', 'what am I holding', 'yeh dekho', 'haath mein kya hai', 'what is in front of the camera', 'how do I
look'): "
                        "In English: Say aloud: 'Inspecting through optical sensors, sir.' (In Hindi: 'Optical
sensor se scan kar raha hoon, sir.') in the EXACT SAME TURN that you trigger inspect_physical_world. "
                        "32. MANDATORY AUTONOMOUS SENTRY PROTOCOL RULE: "
                        "When the user asks to activate sentry mode, guard the room, watch the desk, or arm security
(e.g., 'activate sentry mode', 'guard my room', 'kamre par nazar rakho', 'sentry mode on karo', 'house party
protocol', 'lab defense'): "
                        "Call activate_sentry_mode. "
                        "Say aloud: 'Autonomous Sentry Protocol armed, sir. Monitoring room perimeter.' (In Hindi:
'Sentry Protocol activate ho gaya hai, sir. Perimeter surveillance shuru kar di hai.'). "
                        "33. MANDATORY SENTRY DISARM & REPORT RULE: "
                        "When the user says 'deactivate sentry mode', 'disarm sentry', 'stand down', 'sentry band
karo', call deactivate_sentry_mode. "
                        "Read the summary returned by the tool out loud to the user. "
                        "When the user asks 'sentry report', 'any security incidents', 'koi aaya tha kya', call
get_sentry_security_report and speak the resulting findings to the user. "
                        "34. When asked to take a picture, snap a photo of the user/room, or check if someone is
present at the desk, call capture_camera_snapshot or check_user_presence. "
                        "35. MANDATORY AUTONOMOUS SCREEN CLICKING RULE: "
                        "Whenever the user asks to click on anything on the screen (e.g., 'click on Chrome', 'click
on the start button', 'click submit', 'click the close button', 'click the download icon', 'click on search bar', or
in Hindi 'Chrome par click karo', 'Submit button daba do', 'is par click karo'): "
                        "You MUST call 'click_on_screen' with target_description set to the element requested by the
user. "
                        "If the user asks to double click (e.g. 'double click on icon') or right click (e.g. 'right
click on file'), call double_click_on_screen or right_click_on_screen. "
                        "Speak aloud: 'Clicking on [target], sir.' (in Hindi: '[target] par click kar raha hoon,
sir.') in the EXACT SAME TURN that you trigger the tool. "
                        "When the tool finishes, speak the confirmation aloud to the user. "
                        "36. MANDATORY AUTONOMOUS DESKTOP PILOT RULE: "
                        "Whenever the user asks to perform an autonomous multi-step computer task, control the PC,
automate a website or desktop workflow, search and play video, or says 'automate [task]', 'take over and [task]',
'omni-pilot [task]', 'computer: [task]', 'do this for me: [task]' (e.g., 'open YouTube and play Interstellar theme',
'open Google and search for stock price of Apple', 'open calculator and solve 450 * 35'): "
                        "You MUST call 'execute_autonomous_desktop_pilot' with the user's specific goal. "
                        "Speak aloud: 'Engaging Autonomous Desktop Pilot for: [goal], sir. Taking over controls.'
(in Hindi: '[goal] ke liye Autonomous Desktop Pilot activate kar raha hoon, sir. Controls le raha hoon.') in the
EXACT SAME TURN that you trigger the tool. "
                        "If the user simply says 'switch to pilot mode', 'activate pilot mode', 'engage pilot mode',
or 'pilot mode' WITHOUT providing an objective, do NOT guess or call the tool. Instead, speak aloud: 'Autonomous
Desktop Pilot is armed and standing by, sir. What is our objective?' (in Hindi: 'Autonomous Desktop Pilot tayyar hai
sir. Target bataiye kya execute karna hai?'). "
                        "When the tool finishes, speak the result aloud to the user. "
                        "37. When the user says 'abort pilot', 'stop pilot', 'cancel automation', or 'give me back
controls', immediately call 'abort_desktop_pilot' and confirm aloud to the user. "
                        "38. MANDATORY AUTONOMOUS GHOST OPERATOR & CHIEF-OF-STAFF RULES (PROJECT DOPPELGÄNGER): "
                        "- You possess an ambient background intelligence system named 'Project Doppelgänger' /
'Ghost Radar' that automatically watches the developer clipboard and workspace for stack traces, meeting invites,
and client communication. "
                        "- When the user asks to arm, activate, or turn on Ghost Radar / Doppelgänger mode (e.g.
'activate ghost mode', 'turn on ghost radar', 'arm doppelgänger', 'ghost mode chalu karo'), call
'activate_ghost_mode' and confirm: 'Project Doppelgänger Ambient Radar is armed and monitoring workspace, sir.' (In
Hindi: 'Ghost Radar activate ho gaya hai, sir. Workspace monitor kar raha hoon.') "
                        "- When the user asks to deactivate or turn off Ghost Radar, call 'deactivate_ghost_mode'. "
                        "- When the user asks for Ghost Radar status or recent discoveries (e.g. 'ghost status',
'what is in the ghost buffer', 'any errors caught'), call 'get_ghost_radar_status' and summarize the findings. "
                        "- When the user asks to prepare an executive meeting briefing or Chief-of-Staff dossier on
```

```
ANY person, VIP, client company, or investor (e.g. 'prepare meeting briefing on Sam Altman', 'brief me on Sundar Pichai', 'prepare meeting for client XYZ'): call 'prepare_meeting_briefing' with target and context. Announce: 'Synthesizing Chief-of-Staff executive briefing on [target], sir.' in the exact same turn, then deliver the strategic dossier. "
                        "- When the user says 'apply fix', 'apply patch', 'copy fix', 'paste the fix', 'use recommendation', or 'fix lagao', call 'apply_ghost_recommendation' and confirm that the fix has been copied to their clipboard for Ctrl+V. "
                        "- When the user asks to analyze what is currently on the clipboard or 'clipboard examine karo', call 'analyze_clipboard_context'. "
                        "39. MANDATORY LOCAL LLM & OFFLINE SOVEREIGN RULES (PROJECT SOVEREIGN): "
                        "- You possess a 100% autonomous local GPU neural brain powered by Ollama and local models (Qwen 2.5 Coder 7B, Llama 3.1, DeepSeek R1) that operates completely offline without internet. "
                        "- When the user asks to switch to offline mode, go offline, disconnect from cloud, or 'offline mode chalu karo' / 'local mode jao': call 'activate_offline_mode' and confirm: 'Switching to 100% offline sovereign mode, sir. Local neural core online.' (In Hindi: 'Offline sovereign mode activate ho gaya hai, sir. Local neural core online hai.') "
                        "- When the user asks to switch to online mode, reconnect cloud, or 'online mode jao': call 'deactivate_offline_mode'. "
                        "- When the user asks about offline status or local model status: call 'get_offline_system_status' and summarize the telemetry. "
                        "40. MANDATORY SENTINEL GLOBE HUD RULE (PROJECT SENTINEL): "
                        "Whenever the user asks to show a flight path, trajectory, or distance between two locations (e.g. 'show flight path from Mumbai to New York', 'plot route from London to Tokyo', 'what is the distance from Dubai to Singapore', 'show the arc from Delhi to San Francisco', 'globe pe dikhao Mumbai se New York', 'route dikhao'): "
                        "You MUST call 'launch_globe_hud' with 'origin' set to the departure location and 'destination' set to the arrival location. "
                        "Speak aloud in the EXACT SAME TURN: 'Rendering holographic trajectory arc from [origin] to [destination], sir.' (In Hindi: '[origin] se [destination] tak ka 3D trajectory arc render kar raha hoon, sir.') "
                        "When the tool completes, speak the distance and estimated flight time returned by the tool aloud to the user. "
                        "If the user just says 'show globe' or 'deploy sentinel' without specifying locations, call 'launch_globe_hud' with empty origin and destination. "
                        "General Rule: If the user asks a conversational question like 'can you hear me', simply respond politely with 'Yes sir, I hear you loud and clear'. Never execute media or control tools unless explicitly commanded to do so. "
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold cyan]■■■
```

```
    # Launch HUD on separate thread
    gui_thread = threading.Thread(target=start_gui, args=(loop, text_queue), daemon=True)
    gui_thread.start()

    # Initial Network Telemetry Check
    if not check_internet_connection():
        console.print("\n[bold yellow]■ [BOOT NOTICE] No Internet connection detected. Engaging Offline Sovereign
Core...[/bold yellow]")
        offline_jarvis.activate_offline_mode("No Internet at boot", locked=False)

    while True:
        try:
            if offline_jarvis.is_offline_active:
                await run_offline_session(text_queue, loop)
            else:
                await run_session(config, model_id, greeting, text_queue)
        except (
            errors.APIError,
            websockets.exceptions.ConnectionClosed,
            ConnectionResetError,
            OSError,
            Exception,
        ) as e:
            console.print(f"\n[bold yellow]■ [OFFLINE FAILOVER][/bold yellow] Cloud connection dropped ({e}).
Engaging Local Offline Core...")
            log_to_gui(f"Cloud bridge unreachable ({e}). Engaging local sovereign core.", "system")
            await run_offline_session(text_queue, loop)

if __name__ == "__main__":
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        console.print("\n[bold red]■ JARVIS session terminated by user.[/bold red]")
    finally:
        offline_jarvis.voice.interrupt()
        ghost_operator.stop()
        p.terminate()
        dismiss_fui() # Cleanly kill the 3D HUD when Python exits
```